

前言

ST 公司于 2011 年推出基于 Cortex-M4 内核的 STM32F4 系列产品。相较于采用 Cortex-M3 内核的 STM32F1/F2 系列，STM32F4 系列在性能与功能上实现了显著提升。其最大优势在于集成了硬件浮点运算单元（FPU）与 DSP 指令集，同时主频提升至 168MHz，可提供高达 210DMIPS 的处理能力，因而特别适用于需要进行浮点运算与数字信号处理的场合，被誉为数字信号控制器（DSC），应用前景广阔。

与 STM32F1 相比，STM32F4 的主要优势体现在以下几个方面：

1. **内核架构升级：**STM32F4 采用 Cortex-M4 内核，内置 FPU 并支持 DSP 指令集，而 STM32F1 则采用不包含 FPU 及 DSP 的 Cortex-M3 内核。
2. **系统资源增强：**STM32F4 片内 SRAM 容量高达 192KB，并集成摄像头接口（DCMI）、硬件加密模块（CRYP）、高速 USB OTG、真随机数发生器及 OTP 存储器等高级外设。
3. **外设功能优化：**ADC/DAC 转换速率更高、工作电压更低；配备 32 位定时器与带日历功能的 RTC；IO 口复用能力更强；内置 4KB 后备 SRAM；USART 与 SPI 通信速度显著提升。
4. **运行性能提升：**主频达 168MHz（STM32F1 为 72MHz）；搭载 ART 自适应实时加速器，实现 Flash 零等待访问；FSMC 采用 32 位多重 AHB 总线矩阵，总线访问速度大幅提高。
5. **功耗控制更优：**STM32F40x 系列功耗为 238μA/MHz，低功耗版 STM32F401 更可低至 140μA/MHz，远低于 STM32F1 的 421μA/MHz。

STM32F4 系列涵盖 STM32F40x、F41x、F42x 及 F43x 等多个子系列，各型号之间在软硬件与引脚设计上保持高度兼容，便于产品升级迭代。其中，STM32F42x/F43x 还集成了 LCD 控制器与 SDRAM 接口，适用于需要大屏显示或大容量存储的应用场景。目前该系列已全面量产，供应稳定，尤以性价比突出的 STM32F407 最为市场欢迎。

本手册将以 STM32F407 为例，系统讲解 STM32F4 的各项内部资源，并提供多级实用程序实例。所有示例均已在 MDK 环境下编译通过，用户可直接下载至华清远见 STM32 开发板进行验证。

对于初学者，手册将从零开始手把手教学，涵盖 MDK 环境搭建、工程创建、编译、仿真、下载与调试等全流程，助您快速入门。

实验平台推荐使用华清远见多核心单片机实验箱，用户可直接运行配套光盘中的例程进行验证。若无此实验箱，亦可选用其他类似开发板。若使用自有开发板，只要具备相应外设资源，代码通常可直接移植，仅需根据实际硬件调整底层驱动（如 IO 配置）即可。

第1章 初识 STM32

单片机 (Single-Chip Microcomputer, SCM)，广义上也被称为微控制单元 (Microcontroller Unit, MCU)，是一种在一个集成电路上集成了中央处理器 (Central Processing Unit, CPU)、内存 (RAM、ROM)、输入/输出端口和定时/计数器等核心组件的小型计算机系统。

单片机在各种领域都有广泛的应用，其灵活性和可编程性使其成为许多嵌入式系统的首选控制器。

在家电领域，单片机被用于控制洗衣机、空调、微波炉等家用电器的功能和操作。它们可以实现各种复杂的控制算法，如温度控制、湿度控制、定时器等，提高了家电产品的智能化水平。

在汽车电子领域，单片机被广泛应用于车载电子系统中，如电控单元 (Electronic Control Unit, ECU)、车载娱乐系统、车身控制模块等。它们可以实现引擎的点火控制、燃油喷射控制、车辆稳定性控制等功能，提高了汽车的性能、安全性和舒适性。

在工业自动化领域，单片机被用于控制各种工业设备和机器，如可编程逻辑控制器 (Programmable Logic Controller, PLC)、数控机床、自动化生产线等。它们可以实现生产过程的自动化控制、监控和优化，提高了生产效率和产品质量。

总之，单片机作为一种功能强大、灵活多样的控制器，已经成为现代社会各个领域不可或缺的重要组成部分，推动着科技进步和社会发展。

1.1 ARM Cortex 系列架构

STM32 是基于 ARM Cortex-M 内核的单片机。那么什么是 ARM Cortex？ARM Cortex 系列共有哪些架构呢？

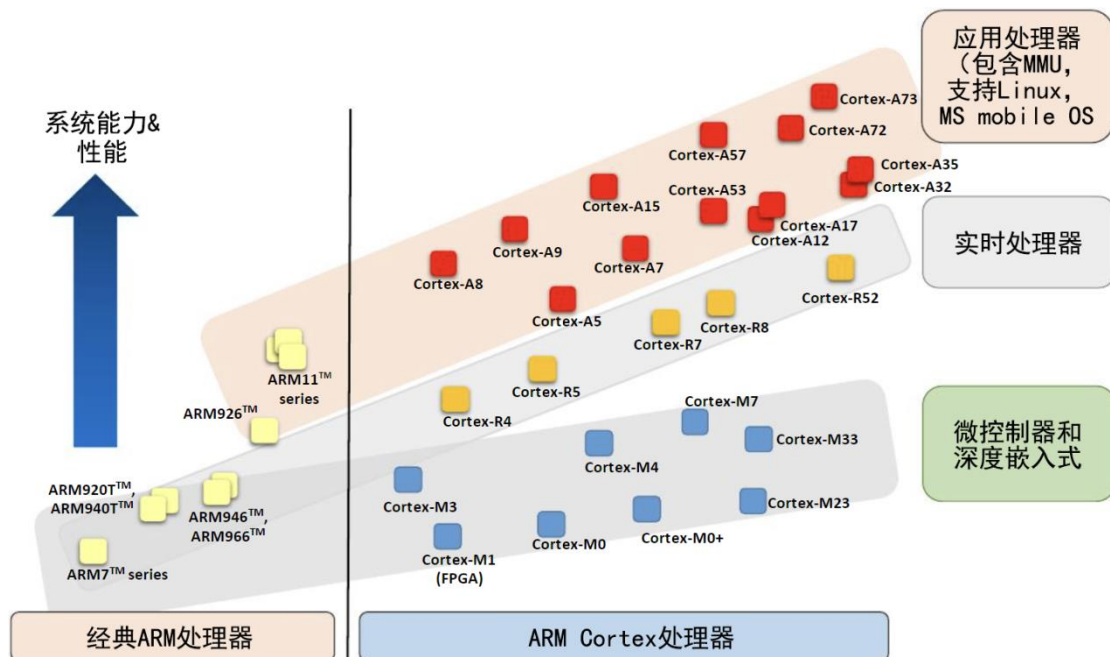
ARM 公司将经典处理器 ARM11 以后的产品改用 Cortex 命名，并分成 A、R 和 M 三类，旨在为各种不同的应用场景提供服务。

ARM Cortex-A 系列处理器主要用于高性能计算和复杂应用，如智能手机、平板电脑、智能电视等，A 代表 Application。这些处理器通常采用了较大的缓存和高性能的指令流水线，以提供出色的性能和能效比。Cortex-A 系列处理器支持多核配置，并且通常配备了高级的特性，如 NEON SIMD 指令集、浮点运算单元等。

ARM Cortex-R 系列处理器专为实时应用设计，如汽车电子系统、工业控制、实时图形处理和无线基站等，R 代表 Real-time。这些处理器具有低延迟和可预测性，能够满足对实时性要求严格的应用场景。Cortex-R 系列处理器通常配备了硬实时特性、错误检测和纠正功能，以确保系统的可靠性和稳定性。

ARM Cortex-M 系列处理器是针对低功耗、成本敏感和资源受限的嵌入式系统而设计的，如传感器、微控制器和物联网设备，M 代表 Microcontroller。这些处理器具有低能耗和小尺寸的特点，但仍提供了足够的性能和功能来满足嵌入式应用的需求。Cortex-M 系列处理器通常具有简化的指令集、低功耗模式和低延迟中断响应，以实现电池寿命和实时性的优化。

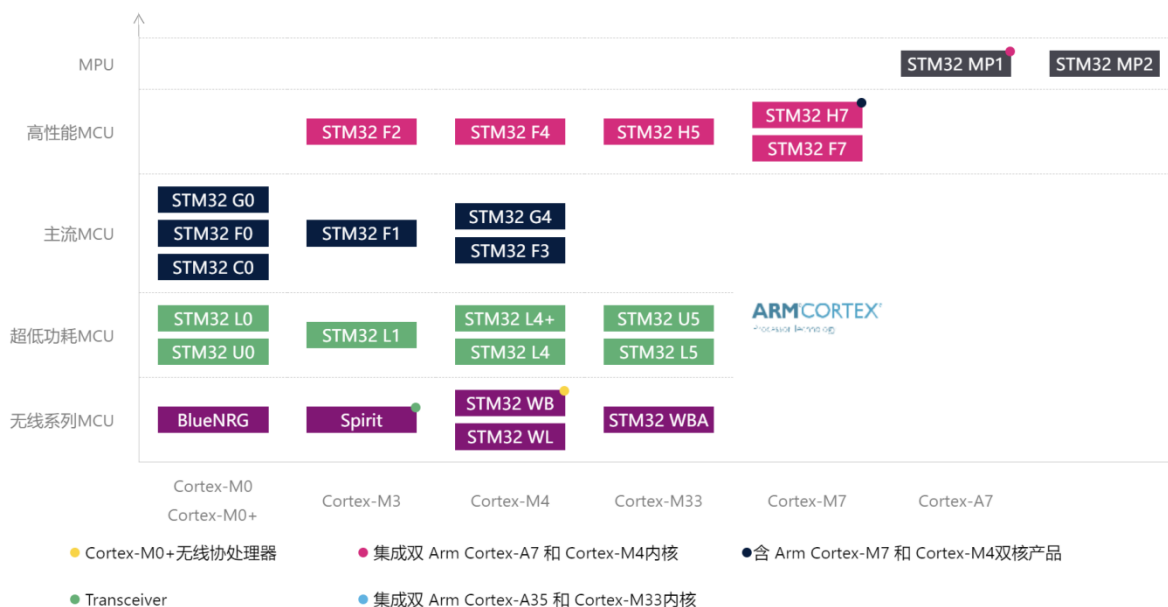
在每个 Cortex 系列处理器下面，还会用数字区分子系列，如 Cortex-A9、Cortex-R8、Cortex-M4 等。ARM 系列处理器性能对比如下图所示。



1.2 STM32 系列 MCU

ARM 公司是一家知识产权 (Intellectual Property, IP) 供应商，它与一般的半导体公司的不同就是不制造芯片且不向终端用户出售芯片，而是通过授权设计方案，由合作伙伴生产出各具特色的芯片。其中，意法半导体 (STMicroelectronics) 就是一家基于 ARM 公司研究的架构和设计方案来生产芯片的半导体公司。STM32 系列 32 位微控制器基于 Arm Cortex-M 处理器内核，由意法半导体公司生产，包括一系列产品，集高性能、实时功能、数字信号处理、低功耗/低电压操作、连接性等特性于一身，同时还保持了集成度高和易于开发的特点。

STM32 具有多种型号，主要分为高性能、主流、超低功耗、无线 4 个系列，如下图所示。



为了便于命名不同型号的 STM32 系列 MCU，STM32 有一套产品型号命名规则，如下图所示。



STM32G070、STM32F103、STM32F405、STM32U575 是 4 个常见的 MCU 型号，下面针对这 4 种型号进行详细介绍。

STM32G0 系列的定位是入门级 32 位 MCU，因此功能较少，价格较低，可满足基础需求，适合成本敏感型应用。其采用 ARM Cortex-M0+ 内核，运行频率高达 64 MHz。STM32G0 系列提供超精确的内部时钟，无须外部振荡器，并拥有全套高性能外设，例如 12 位 ADC、定时器、日历 RTC 和通信外设（例如 I²C、USART 和 SPI）。

STM32F1 系列采用 Cortex-M3 内核，最高主频达 72 MHz。该系列 MCU 具有 16KB ~ 1MB Flash、多种控制外设、USB 全速接口和 CAN。其定位于中低端应用市场，具有性能平衡、丰富的外设、低功耗设计、多种封装和存储容量选择以及丰富的开发生态等特点。

STM32F4 系列是带有 DSP 和 FPU 指令的高性能微控制器。STM32F405 系列 MCU 面向需要在小至 4 mm × 4.2 mm 的封装内实现高集成度、高性能、嵌入式存储器和外设的医疗、工业与消费类应用。STM32F405 提供了工作频率为 168 MHz 的 Cortex-M4 内核（具有浮点单元）的性能。该系列 MCU 采用意法半导体 90 nm 工艺和 ART 加速器，具有动态功耗调整功能，能够在运行模式下和从 Flash 存储器执行时实现低至 238 μA/MHz 的电流消耗。同时，STM32F405 具有创新型外设，包括 2 个 USB OTG（其中一个支持 HS）、专用音频 PLL 和 2 个全双工 I²S、通信接口多达 15 个（包括 6 个速度高达 10.5 Mb/s 的 USART、3 个速度高达 42 Mb/s 的 SPI、3 个 I²C、2 个 CAN 和 1 个 SDIO）、2 个 12 位 DAC、3 个速度为 2.4 MSPS 或 7.2 MSPS（交错模式）的 12 位 ADC、频率高达 168 MHz 的 16 和 32 位定时器、支持 Compact Flash、SRAM、PSRAM、NOR 和 NAND 存储器的灵活静态存储器控制器、基于模拟电子技术的真随机数发生器。

1.3 STM32 固件库

由于 STM32 微控制器涉及大量的寄存器、外设、功能配置等，这些代码很难让用户亲自部署，因此意法半导体公司按照 CMSIS 标准为 STM32 微控制器中各个寄存器、外设、功能配置等操作，编写了比较规范和完备的 C 语言驱动函数。用户在为 STM32 编程时，只需要调用这些官方已经编写好的程序接口即可。这些已经被官方封装好了的，针对特定芯片的，提供了各种功能配置和接口的代码集合叫作固件库。

目前，STM32 的固件库主要有 3 类：标准外设库（Standard Peripheral Library）、HAL 库（Hardware Abstraction Layer Library）、LL 库（Low Layer Library）。

标准外设库是涵盖 STM32 外设的 C 语言库集合，包括所有标准器件外设的驱动。相对于 HAL 库，标准外设库接近于寄存器操作，主要是将一些基本的寄存器操作和外设的配置和功能调用封装成了 C 函数。SPL 库的函数名称通常以“GPIO_”、“USART_”、“SPI_”等前缀开头，便于开发人员快速识别和使用。

SPL 库是基于寄存器的编程模型，开发人员需要直接操作外设的寄存器来配置和控制外设。尽管使用 SPL 库相对于直接操作寄存器来说更加方便，但它仍然需要一定的 STM32 微控制器知识和编程经验。

其特点为：

- 平均优化，适合各种情况；
- 无须进行直接的寄存器操作；
- 对所有外设进行 100%覆盖；
- 更容易调试过程化代码；
- 为复杂中间件（如 USB/TCP-IP/图形/触摸感应）提供扩展。

其局限为：

- 官方不再更新，并鼓励开发者将现有项目迁移到硬件抽象层（HAL）库或者底层驱动（LL）库；
- 较复杂，学习较麻烦，需要对 STM32 微控制器的内部结构和寄存器进行深入的理解，这对于初学者来说可能会增加学习曲线，需要花费更多的时间和精力来掌握；
- 没有通用的 HAL API，影响不同芯片之间的应用的可移植性；
- 中间件库可能在每个系列中不统一；
- 不支持 STM32 L0、L4 和 F7 等新系列的前向兼容。

HAL 库的全称为硬件抽象层，顾名思义，该库将不同芯片间的硬件细节抽象出来，向用户提供一套通用的 API 函数接口，并可以轻松实现将一个型号的 STM32 产品上的程序移植到另一个不同的 STM32 型号上。目前 HAL 库是 ST 公司主推的开发方式。

其特点为：

- 高级别和功能抽象；
- 可轻松地从一个系列移植到另一系列；
- 对所有外设 100%覆盖；
- 集成复杂的中间件，如 USB/TCP-IP/图形/触摸感应/RTOS；
- 可与 STM32CubeMX 配合使用，生成初始化代码。

其局限为：

- 对于嵌入式中的底层 C 语言程序员可能有挑战性；
- 更高的可移植性会导致更大的软件占用空间或更多时间花费在执行适配代码上。

LL 库是在 HAL 库的基础上提供的更底层的库。LL 库提供了对底层寄存器和外设的更直接的访问，并提供了一组低级别的 API 函数。LL 库保留了更多的硬件细节，为开发人员提供了更高级别的灵活性和控制。使用 LL 库，开发人员可以直接编写更底层的代码，实现对微控制器和外设的精细控制。LL 库适用于对性能和资源要求极高，以及对底层硬件控制有特殊需求的应用。

其特点为：

- 高度优化；
- 寄存器级访问；
- 代码表达简洁；
- 严格遵循参考手册；

- 调试接近寄存器级别；
- 可与 STM32CubeMX 配合使用，为 STM32L0/F0/F3/L4 生成初始化代码。

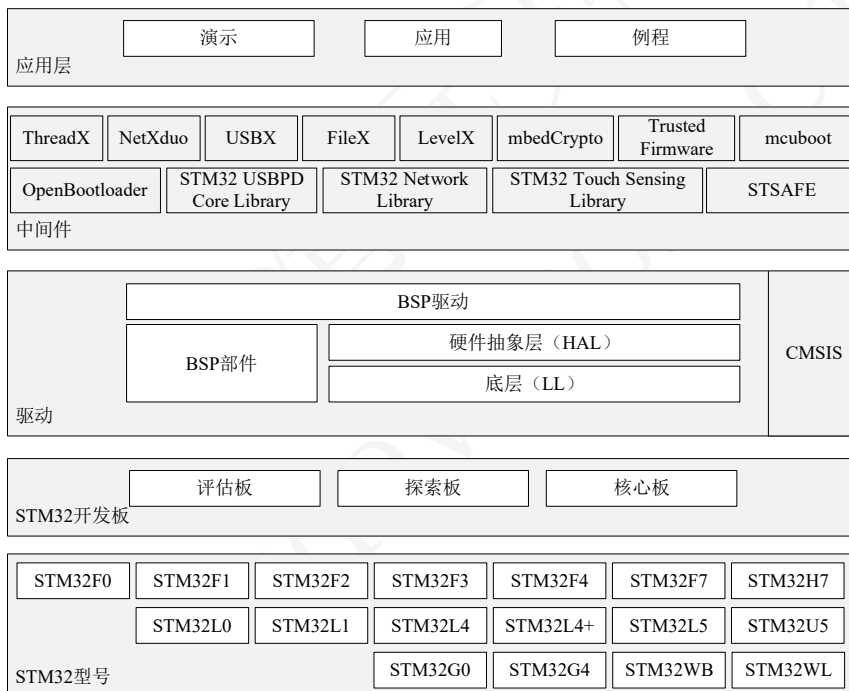
其局限为：

- 针对 STM32 设备特定，不能直接在不同系列之间移植；
- 无法匹配复杂的外设，如 USB；
- 缺乏抽象性意味着开发人员必须理解寄存器级别的外设操作；
- 外设块初始化 API 具有与 SPL（除可用性考虑外）相同的限制。

3 个固件库的对比如下表所示。表中符号“+”的数量越多代表越符合列表头的特点。

		便携性	优化 (Memory 和 Mips)	简单	完备性	硬件覆盖
标准库		++	++	+	++	+++
STM32Cube	HAL 库	+++	+	++	+++	+++
	LL 库	+	+++	+	++	++

本手册主要使用 HAL 库进行讲解。HAL 库可以通过 STM32CubeMX 生成基础配置代码，具有很好的可读性和移植性，同时也是 STM32 官方目前主推的开发方式。HAL 库的接口函数符合 CMSIS 标准，其在整个项目中的位置如下图所示。



第2章 开发环境搭建

在本章中，我们将为你介绍如何搭建 STM32 开发所需的基本开发环境。成功的开发环境搭建是进行 STM32 开发的基础，包括仿真器、编译器和配置工具等软件的安装和配置。本章内容涵盖了 ST-Link 仿真器驱动的安装、MDK 集成开发环境（IDE）的安装，以及 STM32CubeMX 配置工具的安装。通过这一系列步骤，你将能够为后续的 STM32 开发项目奠定坚实的基础。

多核心单片机实验箱资源描述

华清远见多核心单片机实验箱是由华清远见研发中心专为培训教学和项目研发定制的支持多种型号的单片机开发的平台。这里主要介绍该平台上的所有资源，能让开发者快速的找到对应设备的位置，方便开发。



2.1 ST-Link 仿真器驱动安装

为了将开发环境编译生成的文件下载到芯片中并进行调试，需要调试下载工具将电脑和芯片进行连接。JLINK 和 STLINK 是两种常用的用于调试和编程 ARM 微控制器的仿真器，它们都可以通过 USB 接口与电脑连接，实现对目标芯片的内存、寄存器、外设等的访问和控制。ST-LINK 硬件主要由两部分组成：ST-LINK 主控板和连接线。ST-LINK 主控板上有一个 USB 接口，一个 20 针 JTAG/SWD 连接口和 LED 指示灯。USB 接口用于连接电脑，JTAG/SWD 连接口用于连接芯片进行调试和编程。各版本 ST-LINK 如下图所示。

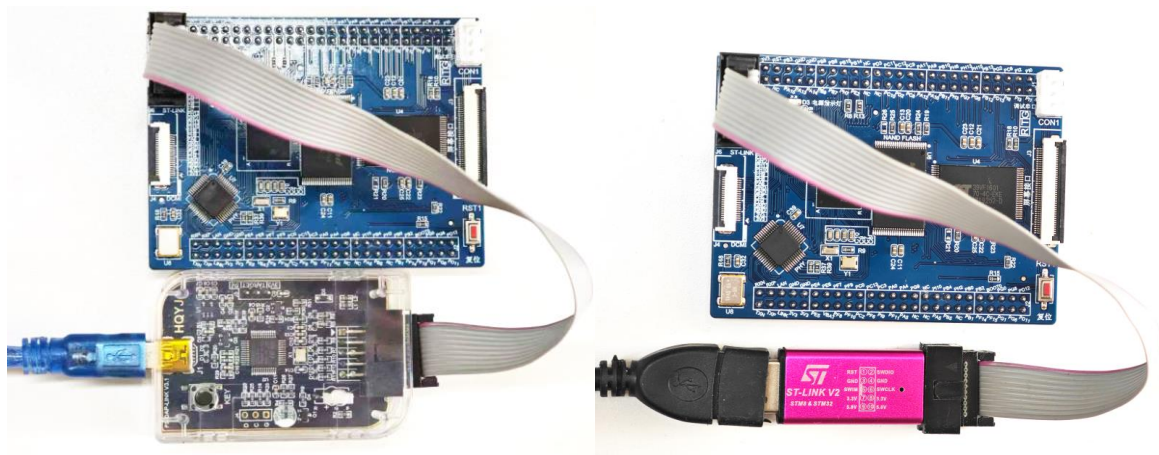


ST-LINK V1 是较老的版本，官方网站上显示已经停产。

ST-LINK V2 是目前比较常见的版本，相比于 V1 有着更高的通信速率。

ST-LINK V3 是针对 STM8 和 STM32 的新一代模块化在线调试兼编程功能的工具。STLINK-V3 包含 3 个版本：STLINK-V3SET、STLINK-V3MINI、STLINK-V3MODS。V3 相较于 V2 有着更高的数据传输速率，同时具备更高灵活性和扩展性，满足定制化需求。

在实验箱中配套的 ST-Link 为以下的样式。

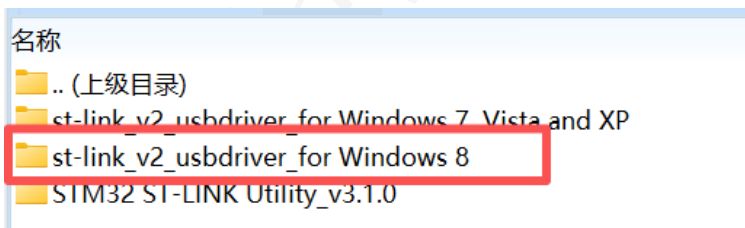


为了在电脑上正常使用 ST-LINK，需要为其安装驱动。驱动程序下载页面如下图所示。



下载并解压后，如果系统是 64 位的，则右击 dpinst_amd64.exe 后选择以管理员身份运行即可安装。如果系统是 32 位的，则右击 dpinst_x86.exe 后选择以管理员身份运行即可安装。

我们可使用资料中附带的驱动程序，位于资料路径下的【工具软件\ST-Link 驱动及工具】。请选择合适的版本安装，一般为下图所示的版本。下面介绍安装该驱动的步骤



将其解压后，打开设备管理器，右击 ST-Link 这个设备，选择“更新驱动程序”。



在打开的窗口中选择“浏览我的电脑以查找驱动程序”。



然后选择解压的文件夹即可。



驱动安装完成后，在电脑上插入 ST-LINK 后，在设备管理器中会显示相应的设备，如下图所示。



2.2 生成工程模板——STM32CubeMX

STM32CubeMX 是一种图形化的配置工具，通过分步过程可以非常轻松地配置 STM32 微控制器和微处理器，以及为 ARM Cortex-M 内核或面向 ARM Cortex-A 内核的特定 Linux 设备树生成相应的初始化 C 代码。在 STM32CubeMX 诞生以前，需要手工一步一步地在 MDK 中引入工程文件，并且手工添加外设配置代码。在使用 STM32CubeMX 以后，我们只需要在软件中勾选 STM32 的相应功能，即可让其自动生成属于 MDK 的配置好外设的工程模板。

2.2.1 STM32CubeMX 的安装

下面介绍该软件的安装过程。相关软件位于目录【工具软件\STM32CubeMX】。



(1) 由于 STM32CubeMX 的使用需要 Java 开发环境，因此需要先去 Java 的官方网站上获取 Java 开发环境的安装包。打开页面后选择下载 Java，下载的文件名类似于 jre-8u271-windows-x64.exe。

(2) 打开下载的 Java 安装包进行安装，如下图所示。



(3) 从 ST 官方网站上获取 STM32CubeMX 安装包（“首页”→“工具与软件”→“开发工具”→“STM32 软件开发套件”→“STM32 配置程序和代码生成器”→STM32CubeMX），如下图所示。在页面的下方选择属于不同系统的安装包进行下载（下载前需注册登录）。

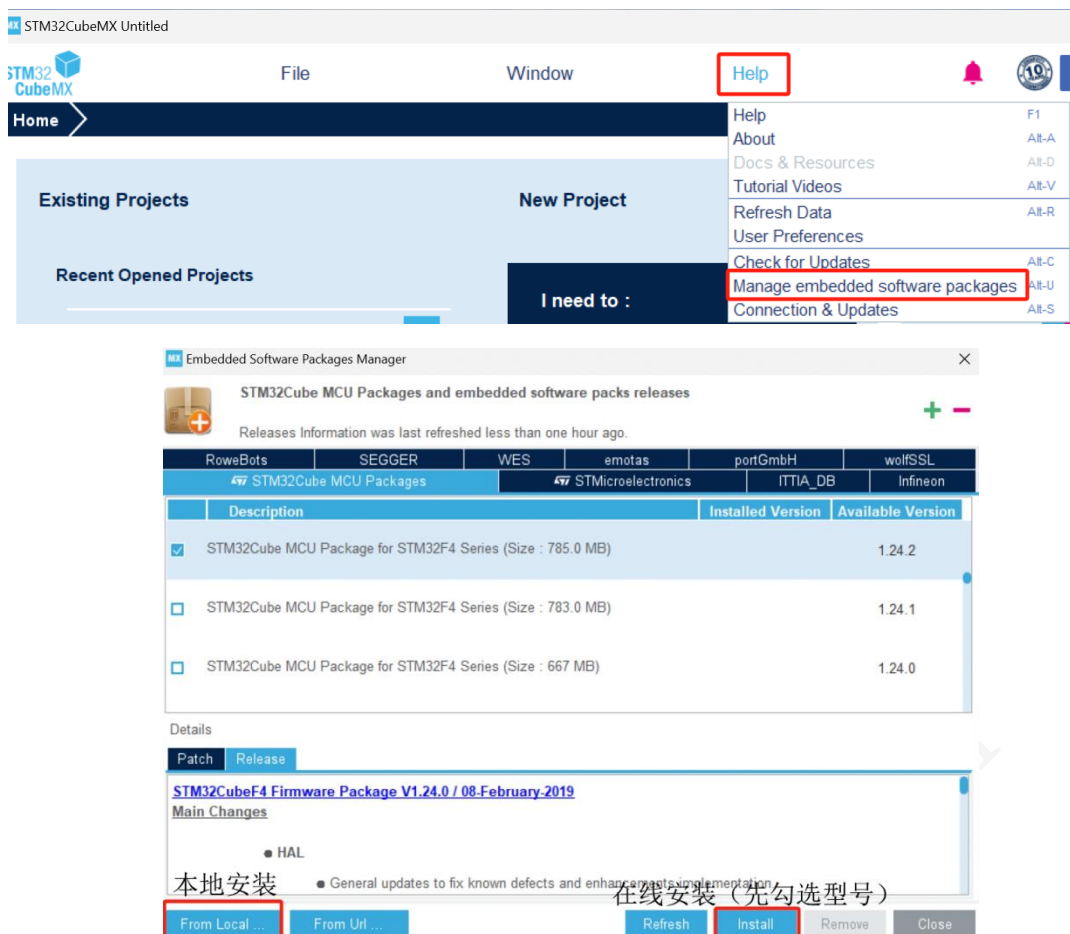


(4) 运行 STM32CubeMX 的安装包进行安装。注意：安装目录不可出现中文。

(5) 安装完成后桌面会出现 STM32CubeMX 的图标。

2.2.2 固件包的安装

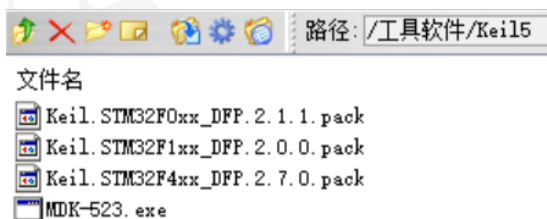
不同的芯片需要使用不同的固件来进行配置，因此在配置 STM32 之前还需要在 STM32CubeMX 中下载相应的固件包。



2.3 MDK-ARM

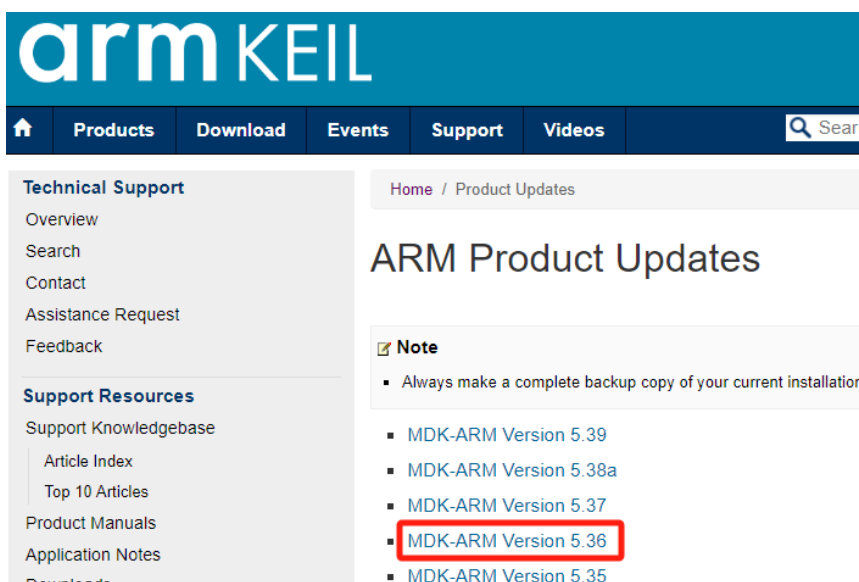
STM32CubeMX 可以对 STM32 的外设进行初始配置，并生成可供集成开发环境打开的工程。为了编写代码并将编写的代码编译成可下载到芯片中的文件，需要利用开发环境对编写的代码进行编译。常见的开发环境有 MDK、EWARM、STM32CubeIDE、GNU 编译器套件（GNU Compiler Collection, GCC）等。

MDK 全称为 Microcontroller Development Kit，即微控制器开发套件。它源自 Keil 公司（后被 ARM 公司收购），现也称为 MDK-ARM。MDK-ARM 软件为基于 Cortex-M、Cortex-R4、ARM7、ARM9 等内核的处理器设备提供了一个完整的开发环境。下面介绍该软件的安装过程。相关的软件位于【工具软件\Keil5】。



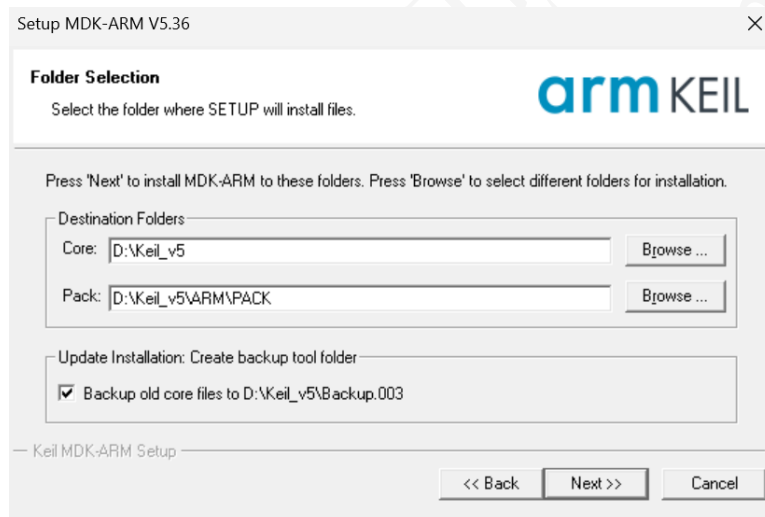
2.3.1 MDK-ARM 的安装

(1) 由于最新的 MDK 不支持 Compiler Version 5 编译器，而利用 STM32CubeMX 生成的工程需要使用此编译器，因此需要下载 5.37 版本以前的 MDK。打开 Keil 官方网站的 ARM Product Updates 页面，在页面中找到 5.36 版本的下载链接并单击打开。



(2) 在打开的页面中，需要填写相关信息才能下载。

(3) 双击安装程序安装即可。注意要安装在全英文目录下，同时设置好程序内核的安装位置和固件包的安装位置。



(4) 安装完成后，桌面上会出现软件图标。



(5) 打开软件后，需要先选择菜单栏的 File→License Management 进行认证才能使用该软件的全部功能。否则编译文件时会受到大小等限制。请咨询官方获取注册码。

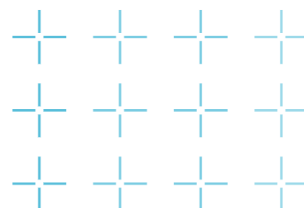
(6) 软件安装完成后，还需要安装针对特定芯片的固件包。该固件包和 STM32CubeMX 中安装的固件包不同，它是应用于 MDK 的，因此需要独立下载安装。打开 ARM Keil 官方网站，在页面上方找到 CMSIS Packs 并打开。

Arm Keil MDK v6

Keil MDK simplifies embedded software development for Cortex®-M and Ethos™-U processors. It offers flexibility through CLI, IDE, or continuous integration workflows.

Learn More

Get the Tools



搜索 STM32F4，找到下图所示的固件包进行下载并安装。

CMSIS Packs

CMSIS-Pack is a distribution format that makes it easy to use software components, device drivers and middleware in [CMSIS](#) development tools including Keil MDK and Keil Studio.

Q STM32F4

Vendor

Content type

☐ Include deprecated packs

Results (8)

Sort by: Relevance

STM32F4xx_DFP Keil

Download version 3.0.0

STMicroelectronics STM32F4 Series Device Support

211 Devices

2.3.2 MDK 软件的使用

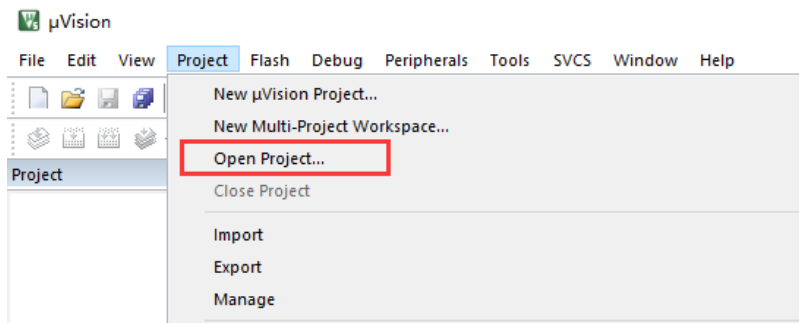
为某个目标系统开发新应用程序时，首先需要创建一个新的项目工程。μVision IDE 的项目开发流程与其他软件开发流程类似，基本步骤如下：

1. **新建工程**：选择目标芯片并从设备库中添加，配置工程的编译器环境。
2. **编写源代码**：使用 C 或汇编语言编写应用程序的源文件。
3. **编译程序**：对源代码进行编译，生成目标应用程序。
4. **调试与修正**：根据编译结果，修改源程序中的错误。
5. **测试与链接**：测试链接生成的应用程序，确保其功能正常。

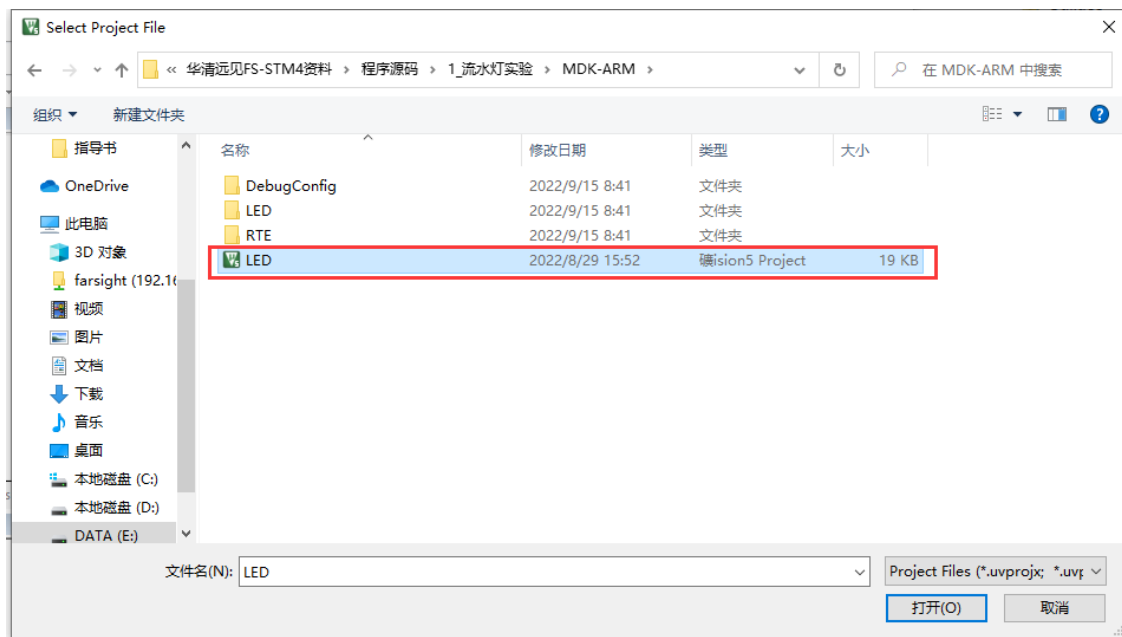
这一流程为开发人员提供了清晰的指导，帮助顺利完成应用程序的开发和调试工作。

导入工程

双击桌面“Keil uVision5”图标运行 Keil 后，选择 Project，点击 Open Project，进行选择工程



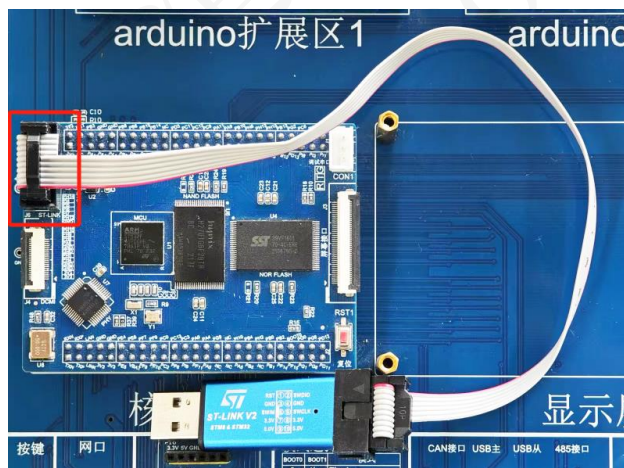
选择到想要导入的工程文件，点击打开



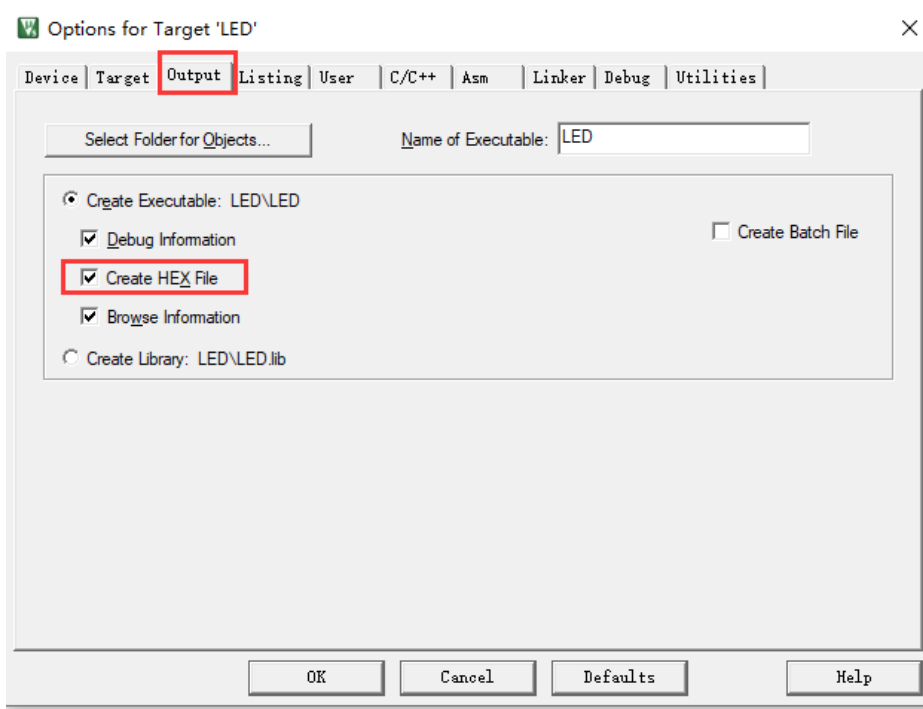
工程基本配置

μVision IDE 允许用户根据目标硬件的实际情况对工程进行配置。通过点击目标工具栏图标  或单击菜单项 Project->Options for Target，在弹出的 Target 页面可指定目标硬件和所选择设备片内组件的相关参数，下图为本示例的相关设置。

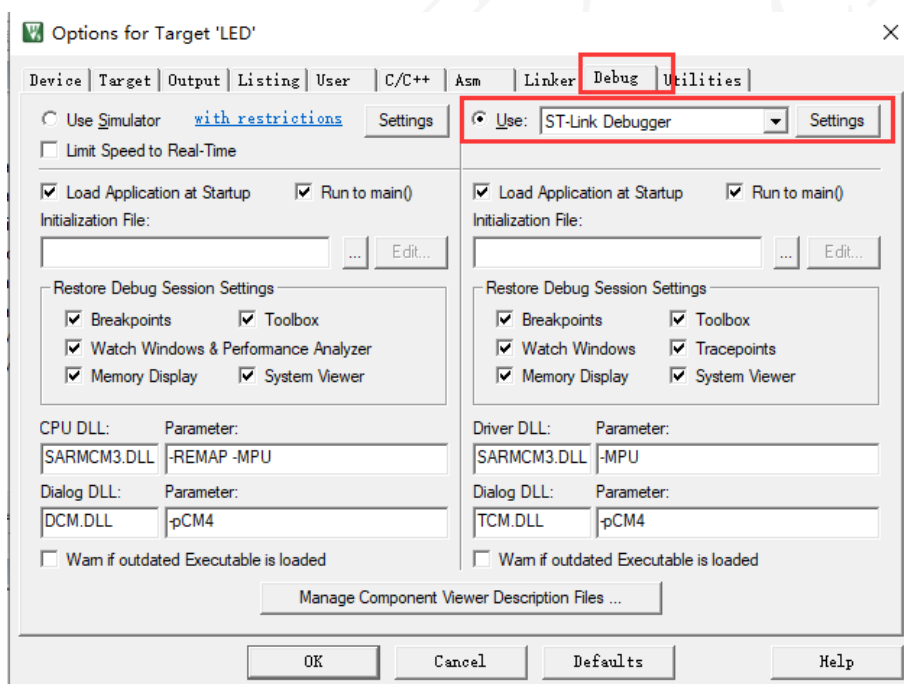
(1) 用 ST-Link 连接电脑和开发板，并上电。与开发板连接的位置如下图所示。另一侧需用 USB 延长线连到电脑上。



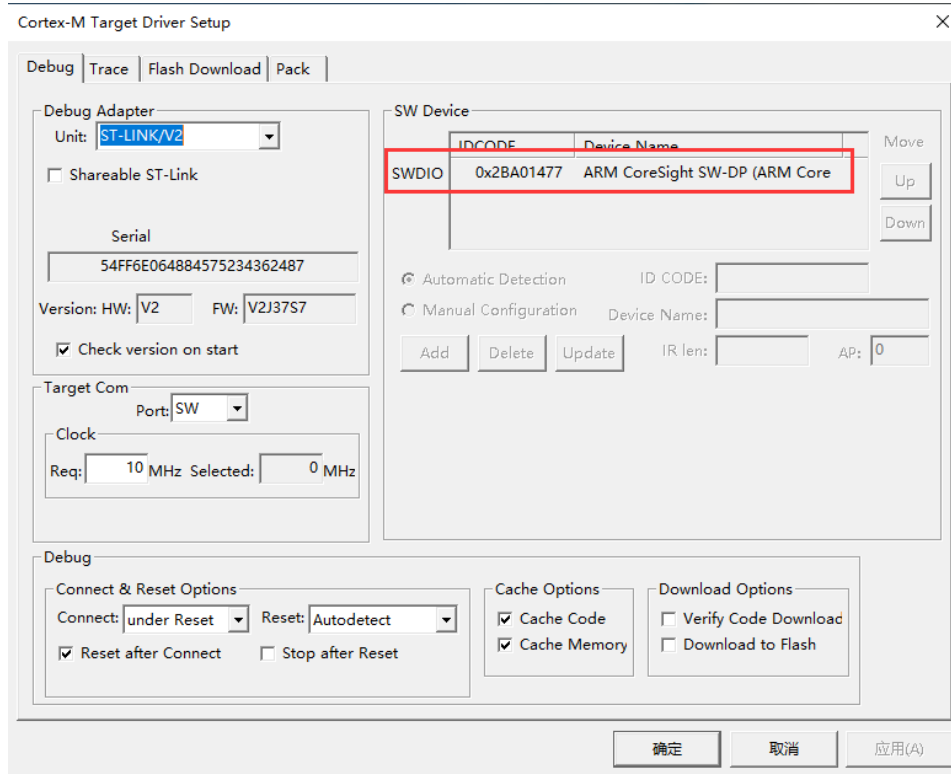
(2) 指定输出文件存放文件夹（创建 HEX 文件）



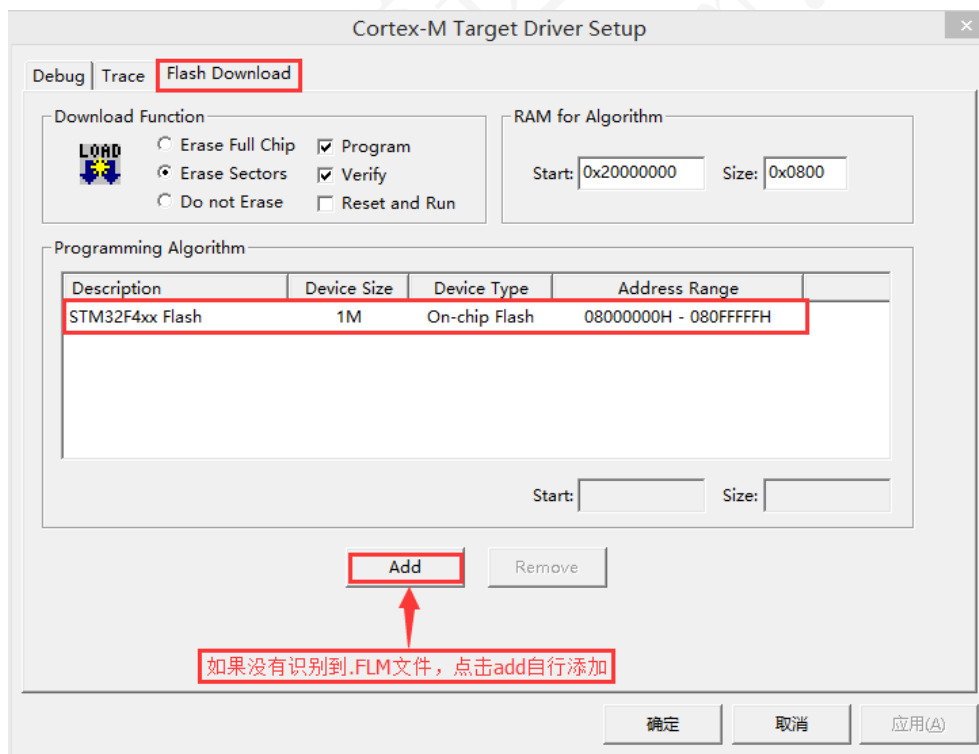
(3) 选择 Debug, 勾选 Use, 选择仿真工具为 ST-Link Debugger, 点击 Setting



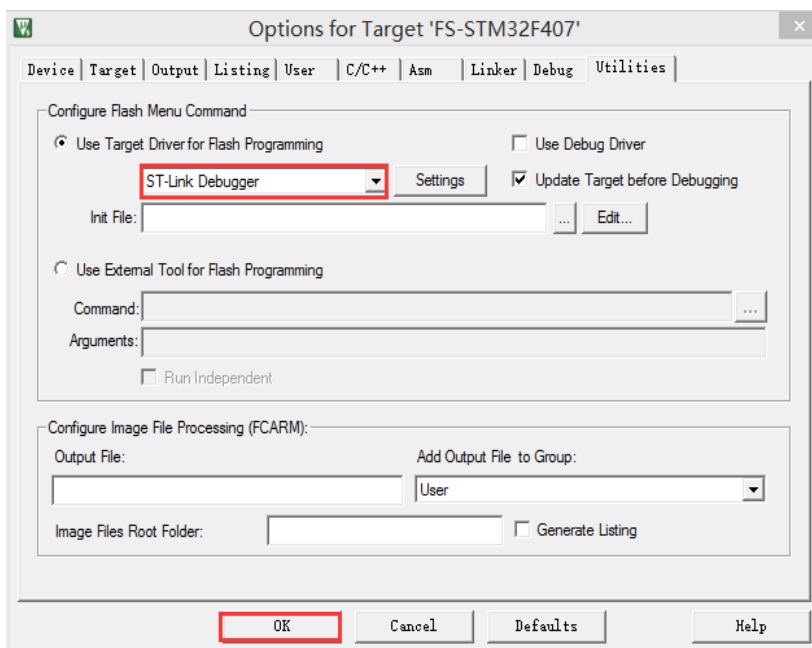
(4) Port 选择 SW, 其它值默认, 可以看到 SWDIO 栏, 自动识别到相关信息 (如果没有识别到信息, 可以重新启动 Keil 软件, 继续打开刚才的界面, 如果还没有识别到信息请检查 ST-link 驱动是否正确安装)



(5) 点击Flash Download, 可以看到已经识别到之前添加的.FLM文件(如果没有请点击add自行添加), 然后点击确定



(6) 选择Utilities, 将目标驱动选择为 ST-Link, 点击 OK 即可



编译、链接

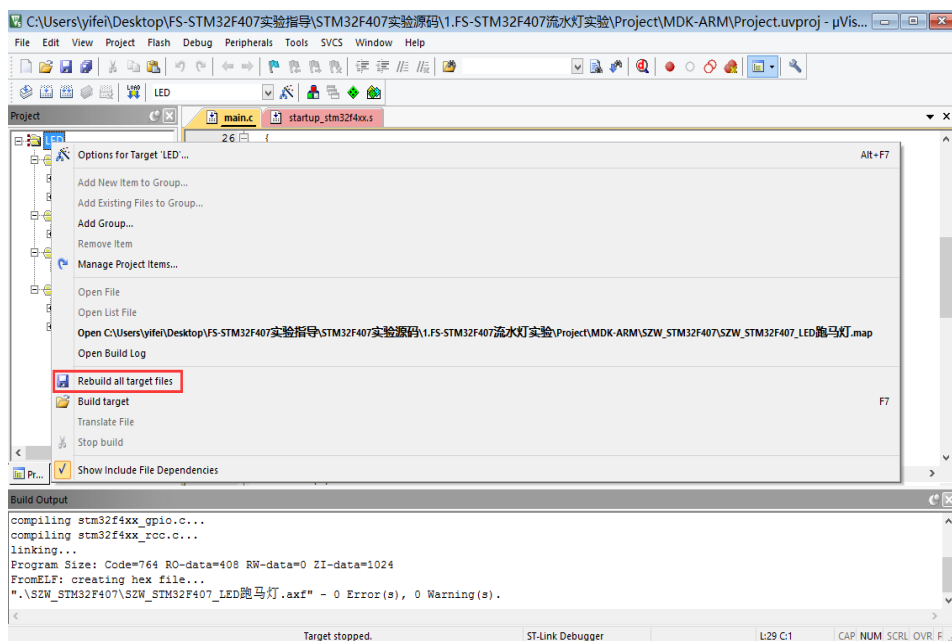
工程已经创建完成，工程文件已经添加到工程目录下，接下来的工作是编译链接工程（可以打开示例

编译一下），单击工具栏中 Build Target  图标可编译链接工程文件。如果源程序中存在语法错误， μ Vision 则会在 Output Window->Build 窗口中显示出错误和警告信息。双击提示信息所在行，就会在 μ Vision 编辑窗口里打开并显示相应的出错源文件，光标会定位在该文件的出错行上，以方便用户快速定位出错位置。



注意：如果要对工程全部重新编译和链接，选中工程名字，点右键选择“Rebuild all target files”。

或者点击按钮 。



下载程序

拨码开关接法：boot0 和 boot1 接法决定了芯片启动方式和从那部分存储介质启动。

BOOT1=x, BOOT0=0 从用户闪存启动，这是正常的工作模式。

BOOT1=0, BOOT0=1 从系统存储器启动，这种模式启动的程序功能由厂家设置。

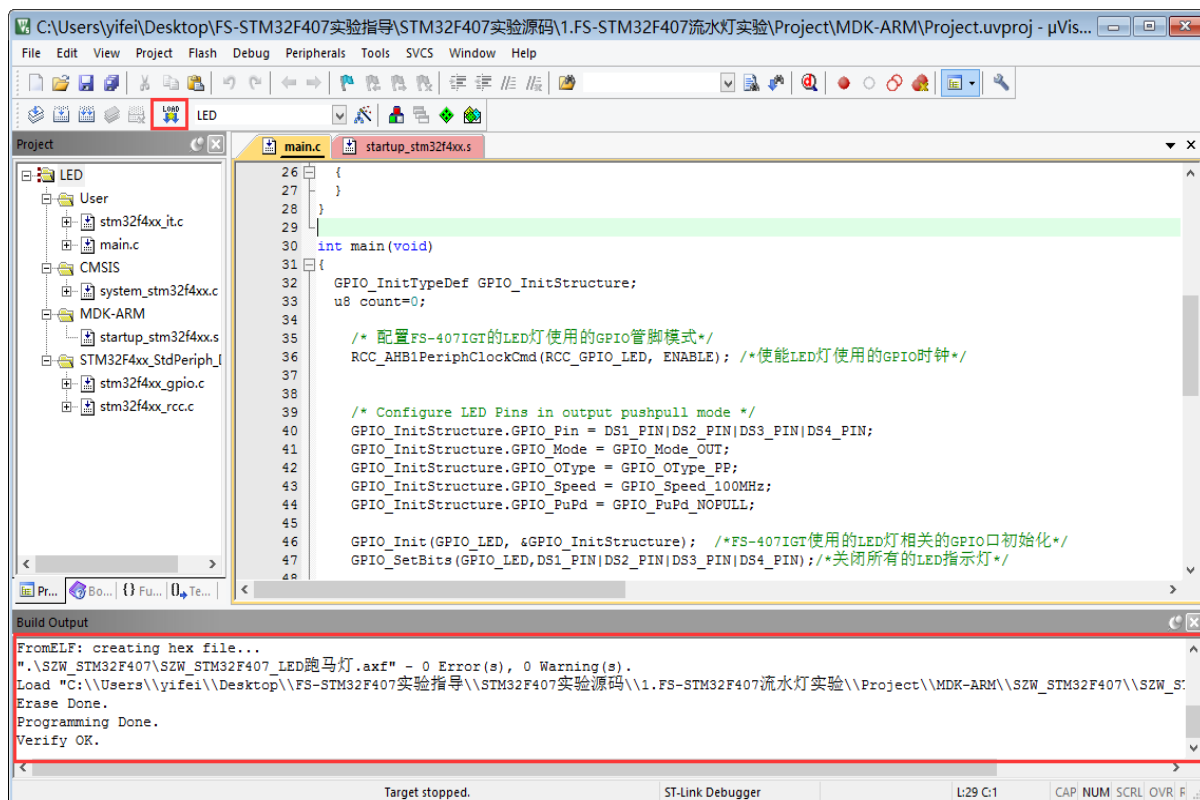
BOOT1=1, BOOT0=1 从内置 SRAM 启动，这种模式可以用于调试。

这里下载前先将 BOOT 0, BOOT 1 拨到 0 状态，如下图所示：



μVision IDE 调试器提供了软件仿真和 GDI 驱动两种调试模式，采用 ST-Link 仿真器调试时，首先将集成环境与 ST-Link 仿真器连接，按照前面的工程基本配置方法对要调试的工程进行配置后，点击 Flash

→ Download 菜单项（或者点击按钮 ）可将目标文件下载到目标系统的指定存储区中。



仿真调试

1. 仿真运行



编译链接或下载完成后，就可使用 μ Vision IDE 的调试器进行调试了。选择 Debug 菜单里的选项 Start/Stop Debug Session 或者点击工具栏里的对应图标进入调试模式。 μ Vision IDE 将会初始化调试器并启动程序运行到主函数。



选择 Debug 菜单中 GO 命令或者点击工具栏里的运行图标以启动程序运行。观察我们的开发板上的 LED1 闪烁。



选择 Debug 菜单里选择 Stop 命令或者点击工具栏里停止图标来中止程序运行。

2. 单步调试和断电



选择 Debug 菜单中的 Insert/Remove Breakpoints 选项，或者单击鼠标右键选择 Insert/Remove Breakpoints 选项，在主函数的开始设置断点。



选择 Debug 菜单或者工具栏里的 Reset CPU 命令对 CPU 进行复位。如果已经中止了程序，可使用 Run 命令启动程序运行。 μ Vision IDE 会在断点处中止程序运行。



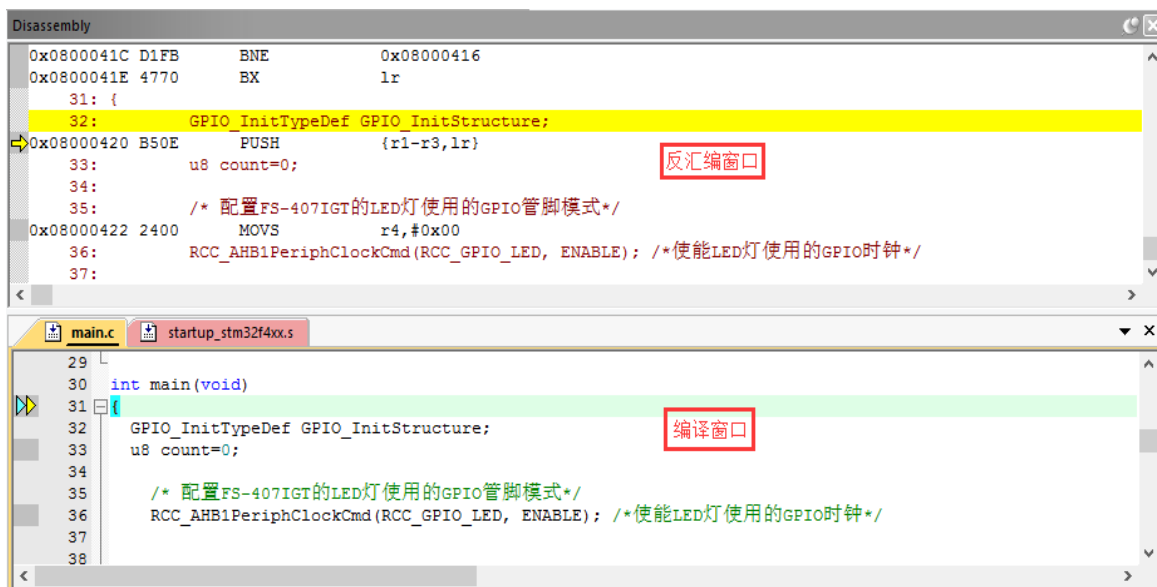
可以使用调试工具栏里的 Step 按钮单步运行程序。当前的指令会用黄色箭头标记出来。将鼠标指针停留在变量上可以观察其相应的值。

这四个按键分别为 Step、Step Over、Step Out、Run to Cursor Line。

注意：请大家在调试时，要逐个试一下这四个工具按钮。在实践中学习，容易理解和掌握。

3. 反汇编

使用 Start/Stop Debug Session 调试程序时，会自动弹出 Disassembly（反汇编）窗口。反汇编窗口（如下图所示）可以将源程序和汇编程序一起显示，也可以只显示汇编程序。



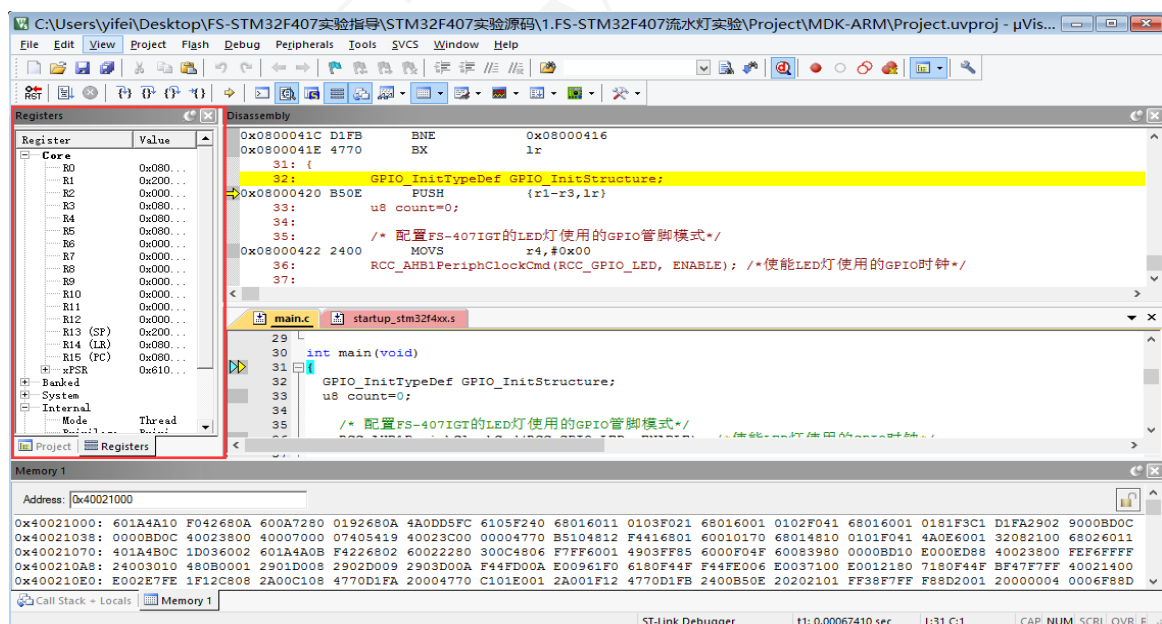
若选择反汇编窗口作为当前窗口，那么程序的执行是以 CPU 指令为单位的而不是以源程序中的行为单位的。可以用工具条上的按钮或快捷菜单命令为选中的行设置断点或对断点进行修改。

4. 逻辑分析仪

µVision IDE 逻辑分析仪可以将指定的变量或 VTREGs 值的变化以图形方式表示出来。是在软件仿真基础上应用的。

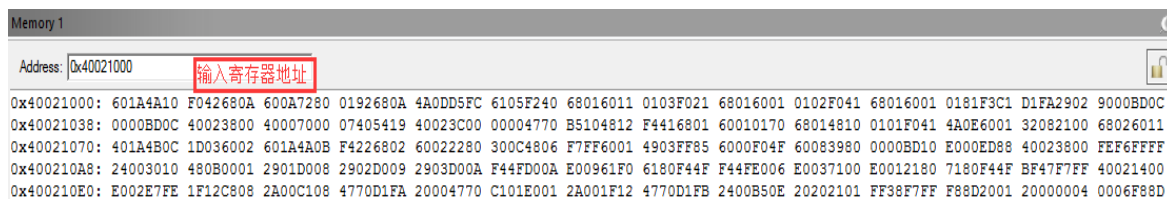
5. 寄存器窗口

是单击菜单栏 View 选择 Registers Window，弹出寄存器显示界面，在 Project Workspace 一> Registers 页中列出了 CPU 的所有寄存器，按模式排列共有四组，分别为 Core 模式寄存器组、System 模式寄存器组、Banked 模式寄存器组、以及 Internal 模式寄存器组，如图所示。在每个寄存器组中又分别有相应的寄存器。在调试过程中，值发生变化的寄存器将会以蓝色显示。选中指定寄存器单击或按 F2 键便可以出现一个编辑框，从而可以改变此寄存器的值。



6. 内存窗口

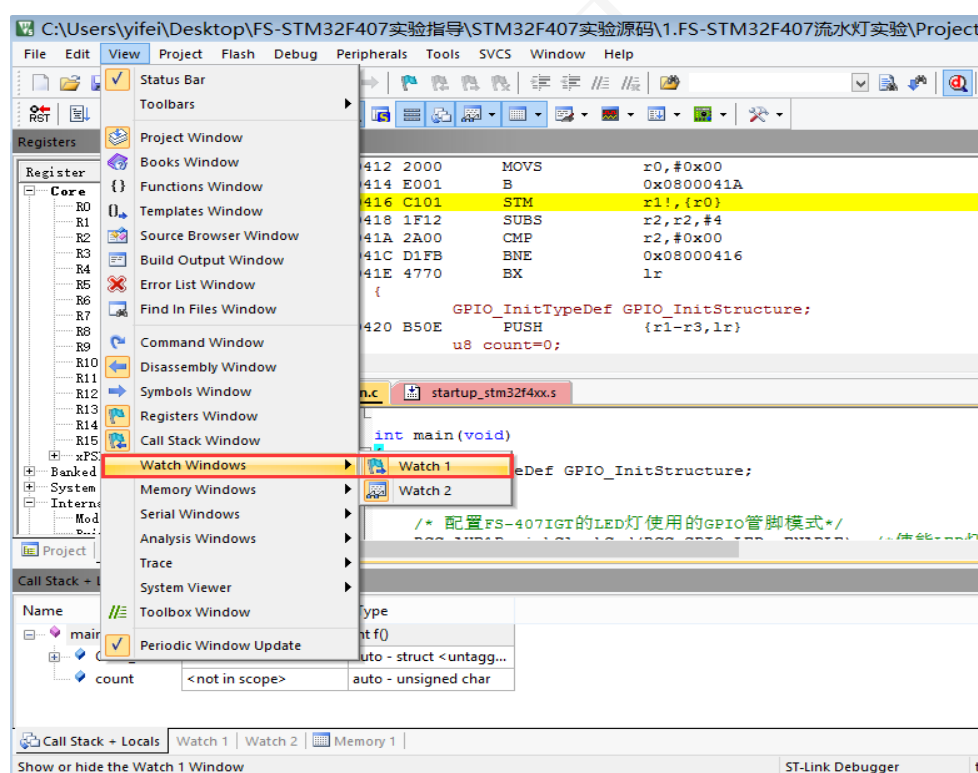
通过工具栏图标弹出内存窗口可以查看存储器内容。内存窗口可以显示不同的存储域内容，最多可将四个不同的存储域显示在不同的页中，窗口中的右键菜单可以选择输出格式。分别为分别为 Memory#1、Memory#2、Memory#3、Memory#4 存储区域。



在 Address 域内，可以输入一个表达式，此表达式的值为所显示内容的地址。在某个单元的值上双击可打开一个编辑框，它允许键入一个新的存储值，即可改变存储内容。使能 View - Periodic Window Update，可以在运行目标程序时更新此窗口中的值。

7. 查看窗口

Watch 窗口用于查看和修改程序中变量的值，并可列出当前函数调用关系。在程序运行结束后 Watch 窗口中的内容自动更新。可能通过设置 View-> Periodic Window Update 来实现程序运行时实时更新变量的值。



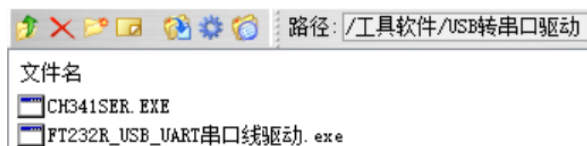
有三种方式可以把程序变量加到 Watch 页中：

- 用鼠标选中<type F2 to edit>，等待 1 秒后再单击一次或按 F2，会出现一个编辑框，在此输入要添加的变量即可；用同样的方法，可以修改已存在的变量。
- 在工作空间中，选中要添加到 Watch 页中的变量，右键点击会出现快捷菜单，在快捷菜单中选择 Add to Watch Window，即可把选定的变量添加到 Watch 页中。

在 Output Window 窗口中，在 Command 页中使用 WS 命令将所要添加的变量添加到 Watch 页中。要修改变量的值，只需选中变量的值，再单击或按 F2 即可出现一个编辑框，进行修改变量的值。要删除变量，只需选中变量，按 Delete 键或用 WK 命令就可以删除变量。

2.4 串口通信工具

为了让电脑和单片机之间进行串口通信，既需要在单片机侧连接串口通信芯片，也需要在电脑上安装对应芯片的驱动程序。相关的驱动程序位于【工具软件\USB 转串口驱动】。



CH340 是一款 USB 转串口的芯片。通过将 CH340 部署到开发板上，可以实现电脑与开发板的串口通信。因此，需要在电脑上安装 CH340 驱动。

打开 WCH 官方产品页面，在页面下方找到驱动程序，选择对应的系统下载即可。这里下载 CH341SER.EXE，因为它与 CH340 驱动兼容。

<https://www.wch.cn/products/CH340.html>



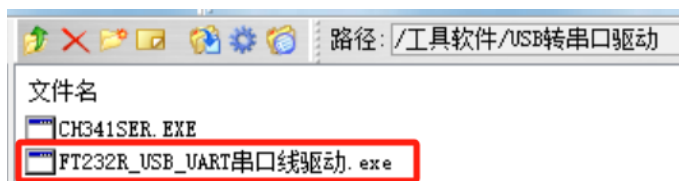
运行下载的安装程序后，单击“安装”按钮。

安装成功后，当利用 USB 线连接含有 CH340 的开发板时，可以看到设备管理器里面能够识别到 CH340，如下图所示。



在开发过程中，可以在电脑上使用串口调试助手软件来与开发板进行串口通信。

另外，我们还需要安装 FT232 驱动。因为一些串口转接线用的是该芯片。



第3章 STM32F407 基础实验

3.1 跑马灯实验

3.1.1 实验目的

使用 GPIO 输出来点亮 LED 灯是一个常见的嵌入式系统实验，也是学习嵌入式系统编程的基础之一。LED 灯可以用作指示设备状态、网络连接状态、电源状态等的指示器。通过本实验，可以了解 GPIO 的基本操作和控制方法，包括配置 GPIO 的输入输出模式、设置引脚状态等，同时可以了解 LED 灯硬件的连接方法和原理。

3.1.2 实验内容

编写 LED 程序，实现按键数码管扩展板上的 4 个 LED 循环闪烁。调试编译好固件程序后，将程序下载到开发板，按复位键观察实验现象。

本实验例程：【实验代码/1_ Marquee】。

3.1.3 GPIO 简介

GPIO 的全称为 General-Purpose Inputs/Outputs，即通用输入输出引脚，是 STM32 微控制器中最重要和最常用的外设之一。其名称“通用”反映了 GPIO 引脚的多功能性和灵活性，它们可以根据需要配置为输入或输出，并能够支持多种应用场景。具体来说，STM32 的 GPIO 可以执行以下几种功能：

输入 (Input)：GPIO 引脚可以接收外部信号并读取其状态。这使得 GPIO 能够与外部设备（如传感器、按钮等）进行交互，监测外部事件或状态变化。

输出 (Output)：GPIO 可以向外部设备发送信号，以控制它们的状态。例如，可以用 GPIO 驱动 LED、马达、继电器等外部组件。

中断 (Interrupt)：GPIO 引脚可以配置为中断触发模式，允许在特定事件发生时发出中断请求，从而及时响应外部事件并执行相应的处理。

复用功能 (Alternative Function)：除了基本的输入和输出功能外，STM32 的 GPIO 引脚还可以实现其他功能，如 UART、SPI、I2C 等通信功能。这使得同一引脚在不同的应用场景下具有不同的用途。

每个 GPIO 引脚通常在 STM32 芯片中以“PAx”、“PBx”、“PCx”等形式命名，其中 A、B、C 表示引脚的组别，而 x 则表示该组内的引脚编号。

不同型号的 STM32 芯片具有不同数量的 GPIO 引脚，且各引脚的复用功能也不相同。因此，在开发过程中，必须查阅芯片的数据手册，以了解每个引脚所支持的功能和配置方式。下图为 STM32F407 的数据手册（注意不是参考手册）。从中可以看到不同封装形式的芯片具有不同数量的引脚。



STM32F405xx STM32F407xx

ARM Cortex-M4 32b MCU+FPU, 210DMIPS, up to 1MB Flash/192+4KB RAM,
USB OTG HS/FS, Ethernet, 17 TIMs, 3 ADCs, 15 comm. interfaces & camera

Features

- Core: ARM 32-bit Cortex™-M4F CPU with FPU, Adaptive real-time accelerator (ART Accelerator™) allowing 0-wait state execution from Flash memory, frequency up to 168 MHz, memory protection unit, 210 DMIPS/1.25 DMIPS/MHz (Dhrystone 2.1), and DSP instructions
- Memories
 - Up to 1 Mbyte of Flash memory
 - Up to 192+4 Kbytes of SRAM including 64-



LQFP64 (10 × 10 mm)
LQFP100 (14 × 14 mm)
LQFP144 (20 × 20 mm)
LQFP176 (24 × 24 mm)



UFBGA176
(10 × 10 mm)

- Up to 140 I/O ports with interrupt capability
 - Up to 136 fast I/Os up to 84 MHz
 - Up to 138 5 V-tolerant I/Os
- Up to 15 communication interfaces
 - Up to 3 × I²C interfaces (SMBus/PMBus)

在该手册的 Table 5 中可以看到每个 GPIO 引脚所能够进行复用的功能列表，如下图所示。

STM32F405xx, STM32F407xx

Pinouts and pin description

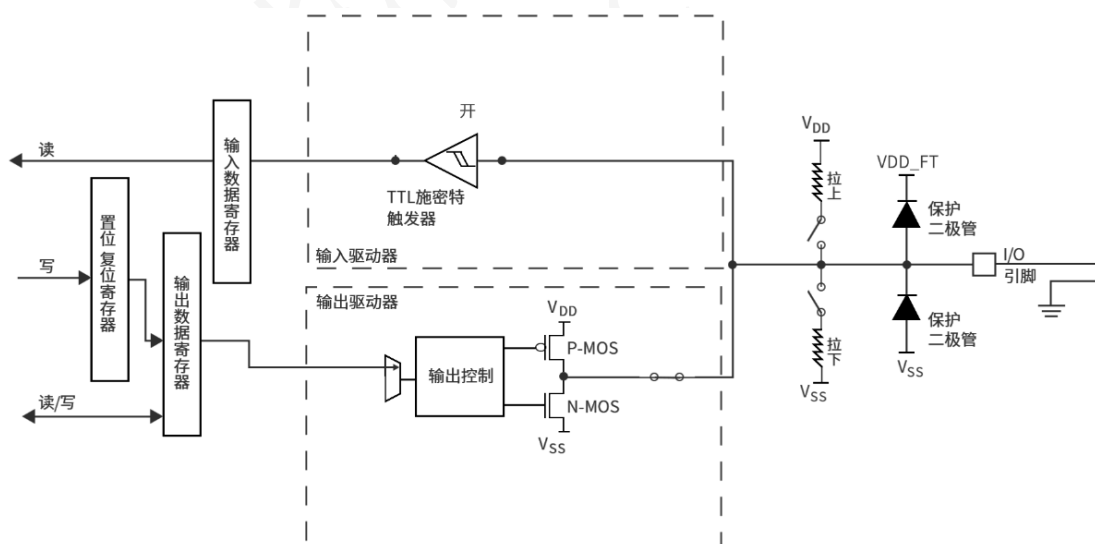
Table 5. STM32F40x pin and ball definitions

Pins					Pin name	I/O Level	Function ⁽¹⁾ after reset	Alternate functions ⁽²⁾
LQFP64	LQFP100	LQFP144	UFBGA176	LQFP176				
-	1	1	A2	1	PE2	FT	PE2	TRACECLK/ FSMC_A23 / ETH_MII_TXD3
-	2	2	A1	2	PE3	FT	PE3	TRACED0/FSMC_A19
-	3	3	B1	3	PE4	FT	PE4	TRACED1/FSMC_A20 / DCMI_D4
-	4	4	B2	4	PE5	FT	PE5	TRACED2 / FSMC_A21 / TIM9_CH1 / DCMI_D6
-	5	5	B3	5	PE6	FT	PE6	TRACED3 / FSMC_A22 / TIM9_CH2 / DCMI_D7
1	6	6	C1	6	V _{BAT}		V _{BAT}	
-	-	-	D2	7	PI8 ⁽³⁾	FT	PI8 ⁽⁴⁾	RTC_AF2
2	7	7	D1	8	PC13 ⁽³⁾	FT	PC13 ⁽⁴⁾	RTC_AF1
3	8	8	E1	9	PC14 ⁽³⁾ -OSC32_IN ⁽⁵⁾	FT	PC14 ⁽⁴⁾	OSC32_IN

总体而言，STM32 的 GPIO 引脚因其高度的灵活性和多功能性，广泛应用于各种嵌入式系统中，满足了各种输入输出需求。

3.1.4 GPIO 的推挽输出模式

GPIO 推挽输出模式（Push-Pull Output Mode）是 STM32 微控制器 GPIO 的一种常见输出模式。在推挽输出模式下，可控制引脚状态为高电平或低电平。这是由输出数据寄存器、输出控制器和 P-MOS 管、N-MOS 管配合实现的，如下图所示。其内部可不连接上拉或下拉电阻，也可连接上拉或下拉电阻以保证抗干扰能力和稳定性。同时，输入线路的施密特触发器也处于打开状态，可以读取引脚上的电平。



当输出数据寄存器对某个引脚输出 0 时，0 会经过输出控制器变为高电平，使得 P-MOS 管关闭，N-MOS 管打开，这样，引脚就会输出低电平。

反之，想让引脚输出高电平时，数据寄存器对输出控制器输出 1，1 会经过输出控制器变为低电平，使得 P-MOS 管导通，N-MOS 管关闭。

尽管在推挽输出状态下引脚由 V_{DD} 通过 P-MOS 管输出高电平，具有一定的驱动能力，但该驱动能力并不高，能够输出的电流在 20 mA 左右（需要查询芯片数据手册），如下图所示。

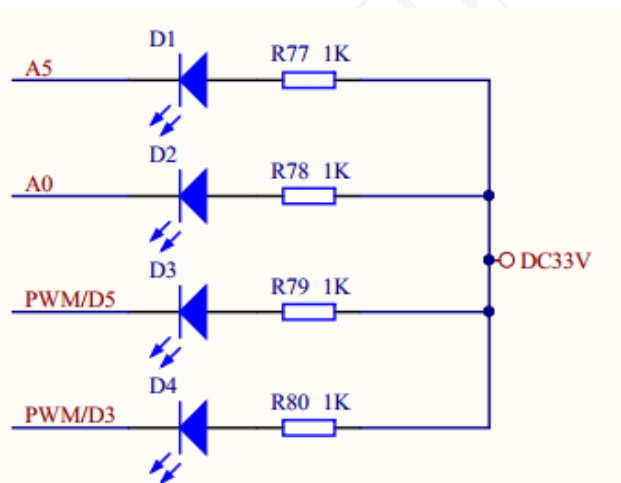
Table 30. Current characteristics

Symbol	Ratings	Max	Unit
ΣI_{VDD}	Total current into sum of all V_{DD} power lines (source) ⁽¹⁾	200	mA
ΣI_{VSS}	Total current out of sum of all V_{SS} ground lines (sink) ⁽¹⁾	200	
I_{VDD}	Maximum current into each VDD power pin (source) ⁽¹⁾	100	
I_{VSS}	Maximum current out of each VSS ground pin (sink) ⁽¹⁾	100	
I_{IO}	Output current sunk by any I/O and control pin	20	
	Output current sourced by any I/O and control pin	20	
$\Sigma I_{(PIN)}$	Total output current sunk by sum of all I/Os and control pins ⁽²⁾	120	
	Total output current sourced by sum of all I/Os and control pins ⁽²⁾	120	
$I_{INJ(PIN)}^{(3)(4)}$	Injected current on FT_xx, TT_xx, RST pins	-5/+0	
$\Sigma I_{INJ(PIN)} $	Total injected current (sum of all I/Os and control pins) ⁽⁵⁾	±25	

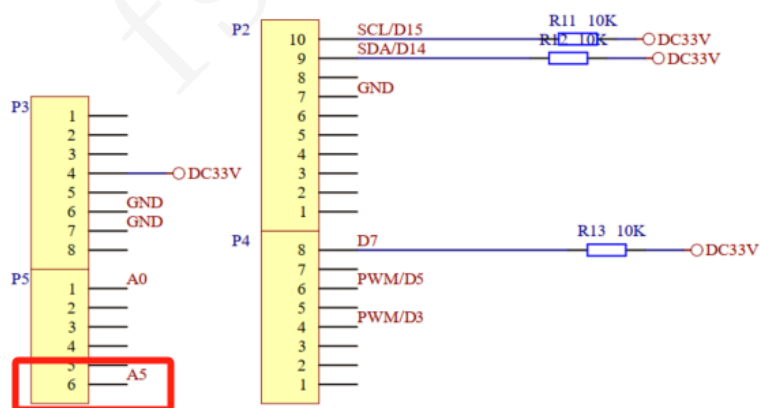
3.1.5 硬件原理

本实验需要控制 LED 灯亮灭。只要是对硬件操作，就要首先查看原理图，查看外设是与单片机的哪个引脚相连。

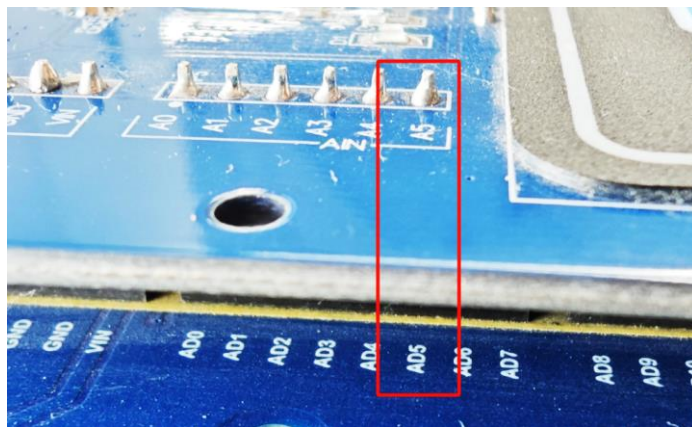
打开实验原理图中的按键数码管扩展板原理图.pdf，找到下图所示位置，由下图可知，当 A5 为低电平时，D1 亮。



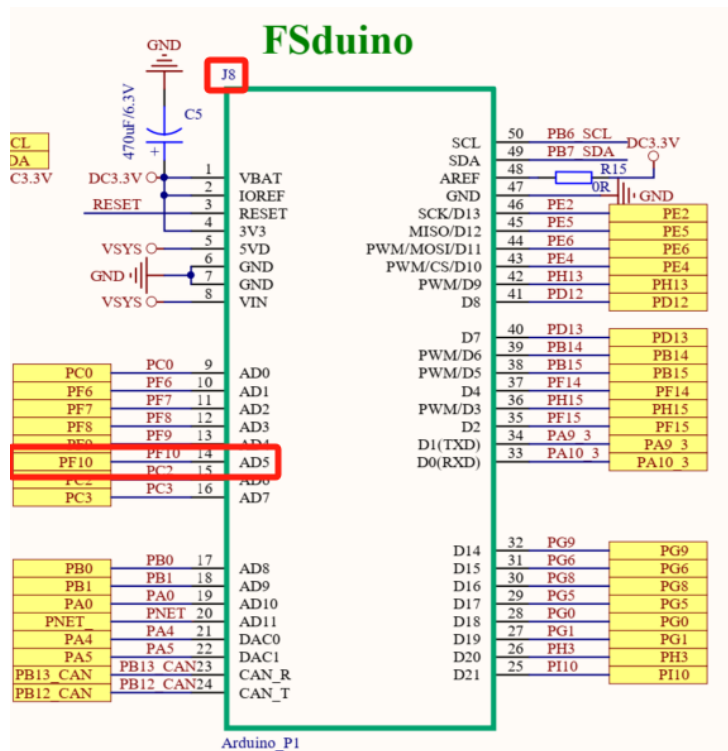
查看按键数码管和实验箱大底板的接口，确定 A5 的位置。从下图可以看出，A5 的连接处是 P5 的编号 6。



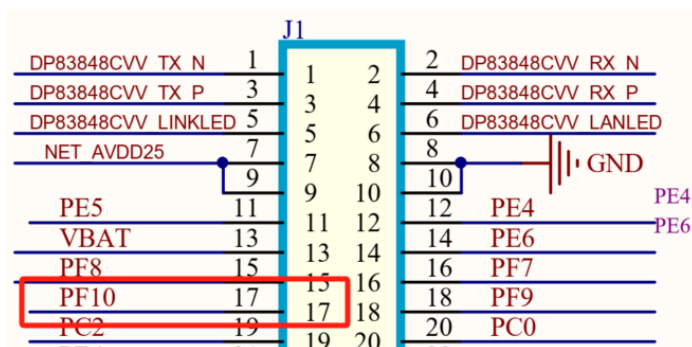
在实验箱上观察该引脚连接了大底板上的哪个引脚。



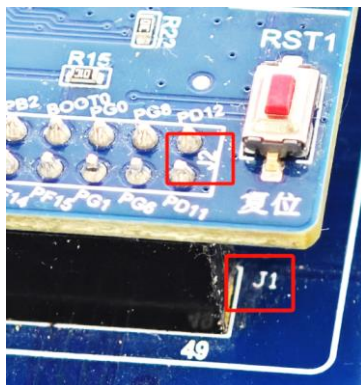
可以看到扩展板的 A5 引脚连接到了大底板的 AD5 引脚上。数码管扩展板位于 J8 插座上。打开底板原理图.pdf，找到 J8 插座的 AD5 所在位置，可以看到其连接的是 PF10 引脚。



该引脚标号只是原理图上的标号，不代表实际的单片机上的真实引脚编号。找到该标号连接到了与核心板连接的引脚插座，如下图所示。可知其位于 J1 插座的 17 号引脚。



通过观察实验箱底板，可知大底板的 J1 插座与核心板的 J2 插座相连。



打开原理图目录下的 FS_STM32F4_核心板.pdf，找到 F4 核心板和底板的接口。

J2			
DP83848CVV TX N	1	1	2 DP83848CVV RX N
DP83848CVV TX P	3	3	4 DP83848CVV RX P
DP83848CVV LINKLED	5	5	6 DP83848CVV LANLED
DC33V	7	7	8
	9	9	10
FSMC A21	11	11	12 FSMC A20
VBAT	13	13	14 FSMC A22
PF8	15	15	16 PF7
PF10	17	17	18 PF9
PC2	19	19	20 PC0
PF6	21	21	22 PC13
PI9	23	23	24 PC3
PF11	25	25	26 PA0
PA5	27	27	28 PA4
	29	29	30 FSMC NE2
	31	31	32

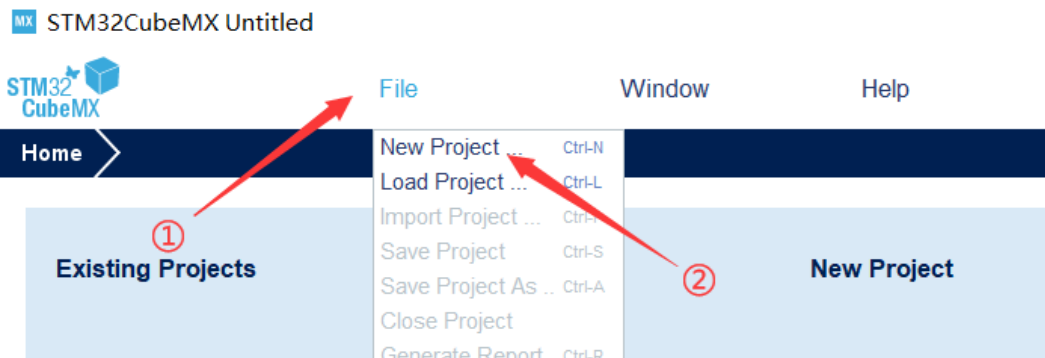
最终确定，我们要控制的 STM32F407 芯片的引脚为 PF10

UIC			
PA0 N3	PA0/WKUP	PG14	A7 PG14
PA1 N2	PA1	PG13	A8 PG13
PA2 P2	PA2	PD2	D12 PD2
PA3 R2	PA3	PD3	D11 PD3
PA4 N4	PA4		K2 PF6
PA5 P4	PA5		K1 PF7
PA6 P3	PA6		L3 PF8
PA7 R3	PA7		L2 PF9
PA8 F15	PA8		L1 PF10
PA9 E15	PA9		R7 PF11
PA10 D15	PA10		
PA11 C15	PA11		H14 PG8
PA12 B15	PA12		J14 PG7
PA13 A15	PA13		B9 PG11
PA14 A14	PA14		R7 PG15
PA15 A13			

同理可以知道另外 3 个 LED 灯连接的引脚为 PC0、PB15、PH15。

3.1.6 STM32CubeMX 配置

- (1) 打开 STM32CubeMX->点击左上角的 File->点击 New Project



- (2) 选择芯片，查看芯片的型号，在搜索框输入芯片型号，本实验中选用的芯片是 STM32F407IGH6

MCU/MPU Selector Board Selector Example Selector Cross Selector

MCU/MPU Filters

Commercial Part Number **STM32F407IGH6**

PRODUCT INFO

- Segment >
- Series >
- Line >
- Marketing Status >
- Price >
- Package >
- Core >
- Coprocessor >

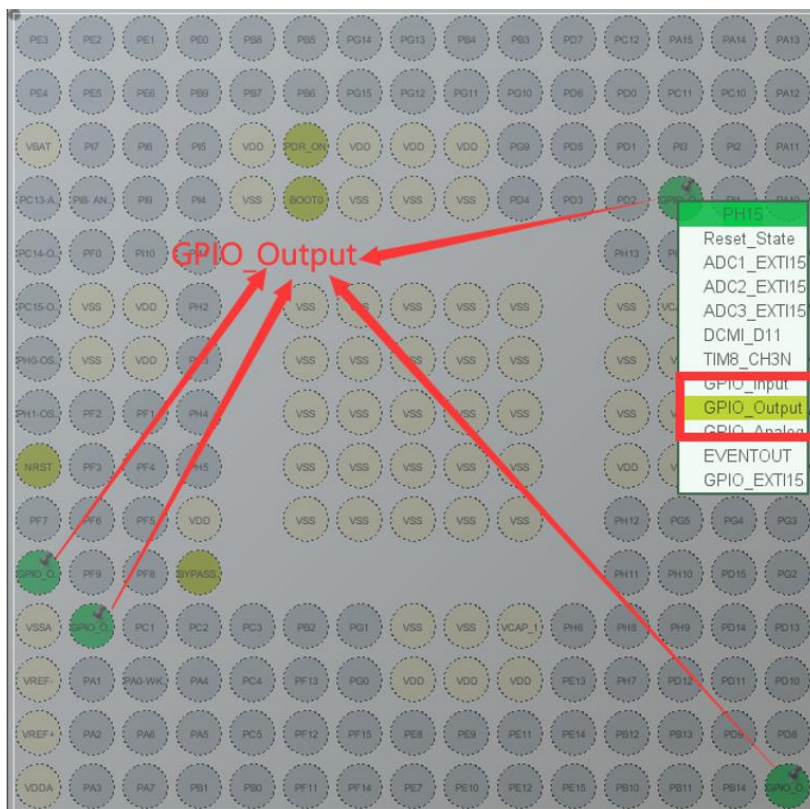
Features Block Diagram Docs & Resources

MCUs/MPUs List: 3 items

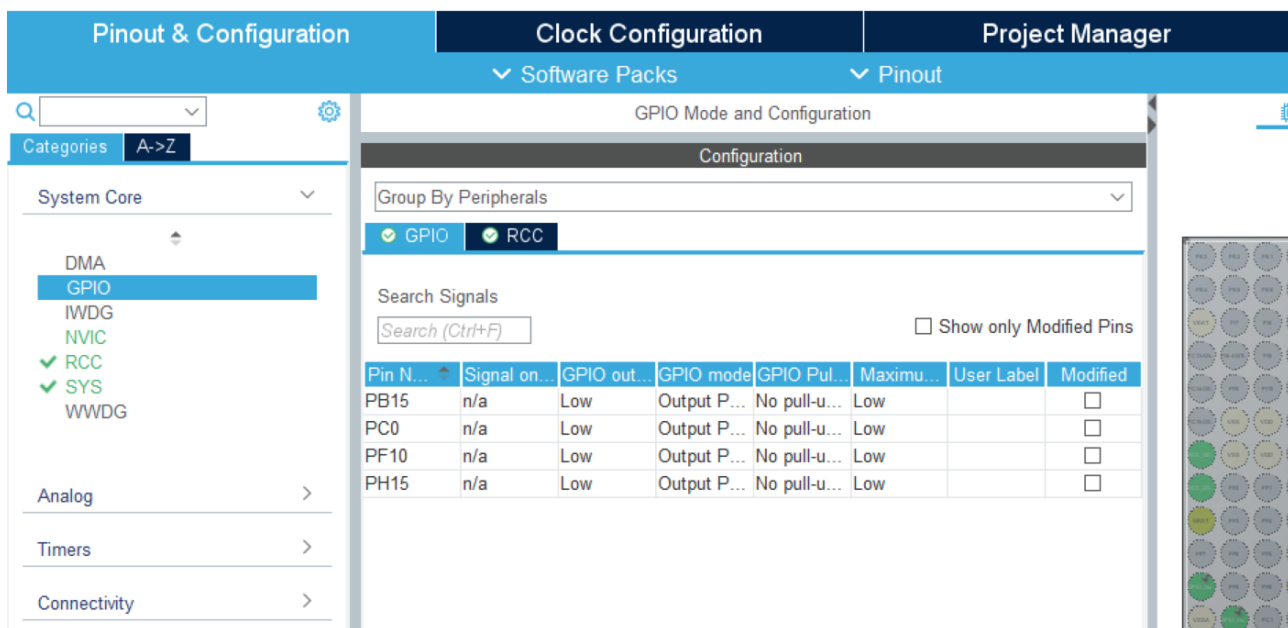
	Part No	Reference	Marketing Status	Unit Price	Unit
★		STM32F407IGHx	Active	6.6667	Uf
★	STM32F407IG	STM32F407IGHx	Obsolete	NA	Uf
★		STM32F407IGHx	Active	6.6667	Uf

双击

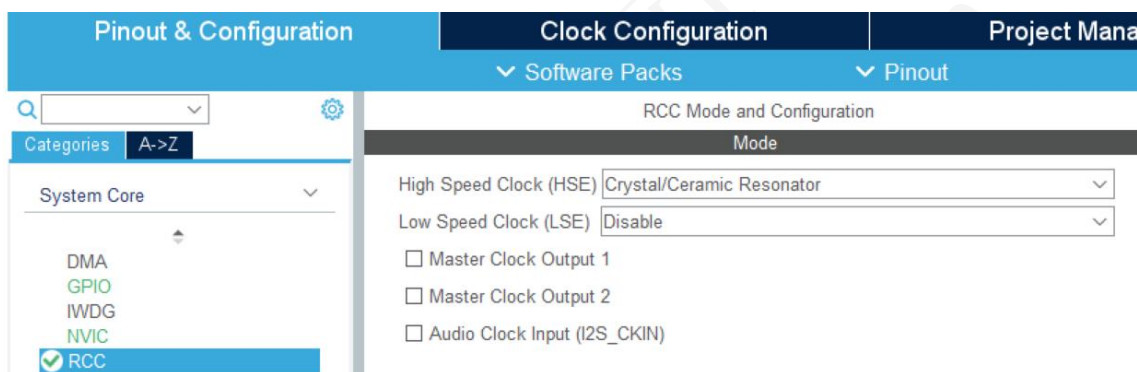
(3) 配置 PF10/PC0/PB15/PH15 为 GPIO_Output 模式，使用默认的 GPIO 即可配置即可



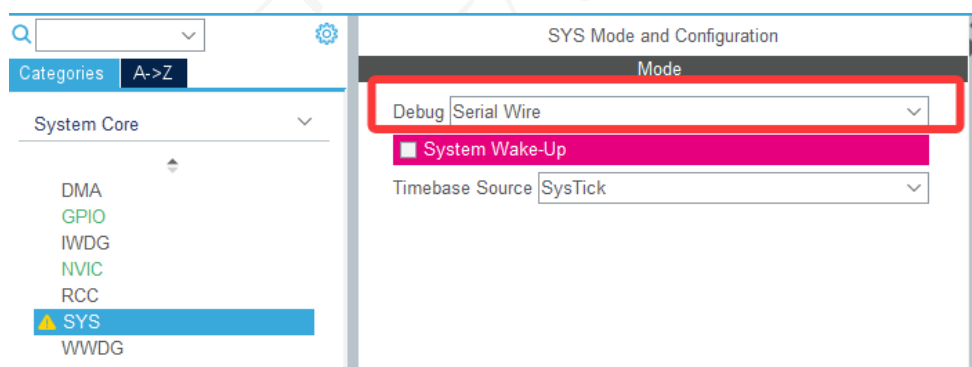
配置完后再 GPIO 功能页即可看到这四个引脚的配置情况。都为 Output Push Pull（推挽输出）模式，默认电平为低电平，无上下拉电阻，输出速度为低。



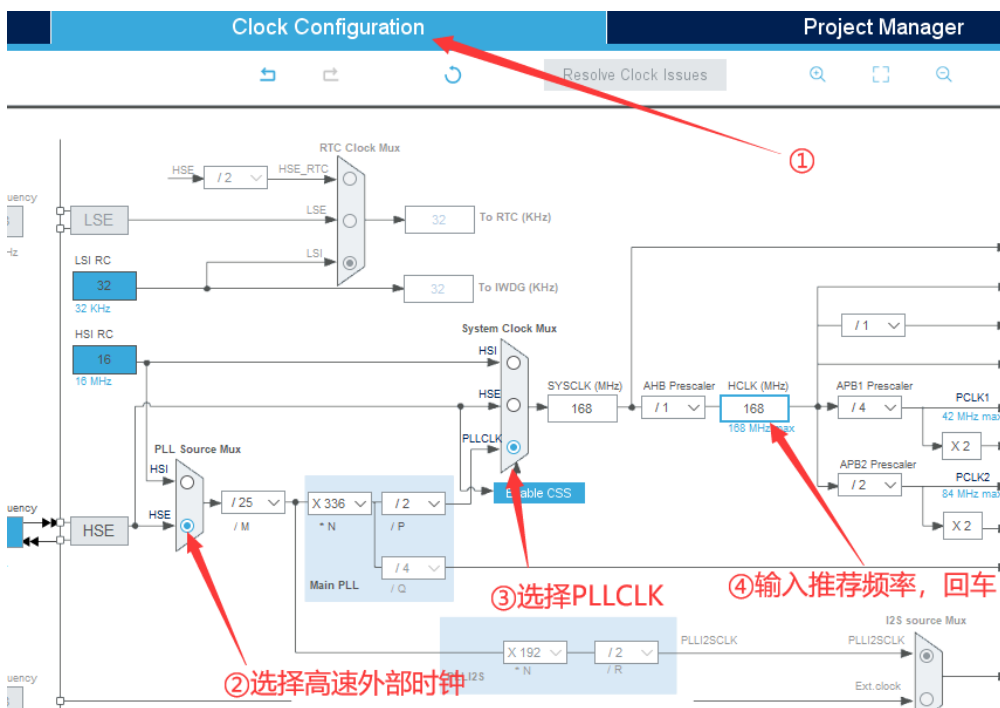
(4) 配置 RCC，点击左侧的 RCC，找到 High Speed Clock (HSE)，点击选择 Crystal/Ceramic Resonator。



(5) 将调试模式修改为 SWD 模式：

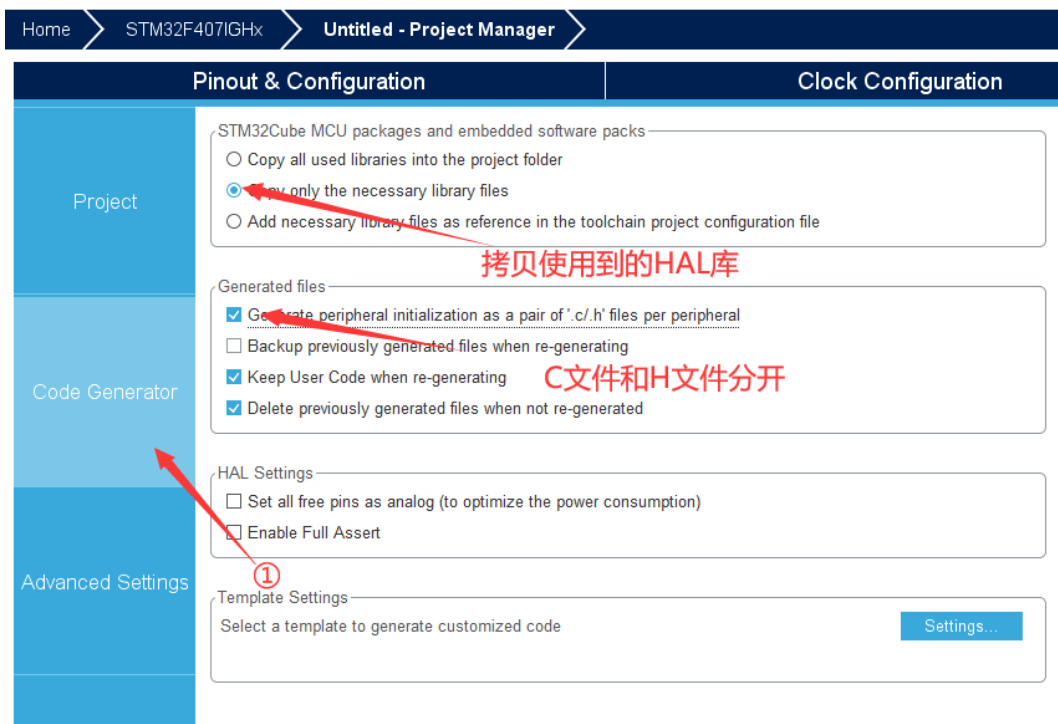


(6) 点击上方的 Clock Configuration，按下图所示配置时钟树。因为核心板上外接的晶振是 25MHz 的，时钟树上默认外接的高速晶振也是 25MHz 的，所以不用改变外接晶振频率。

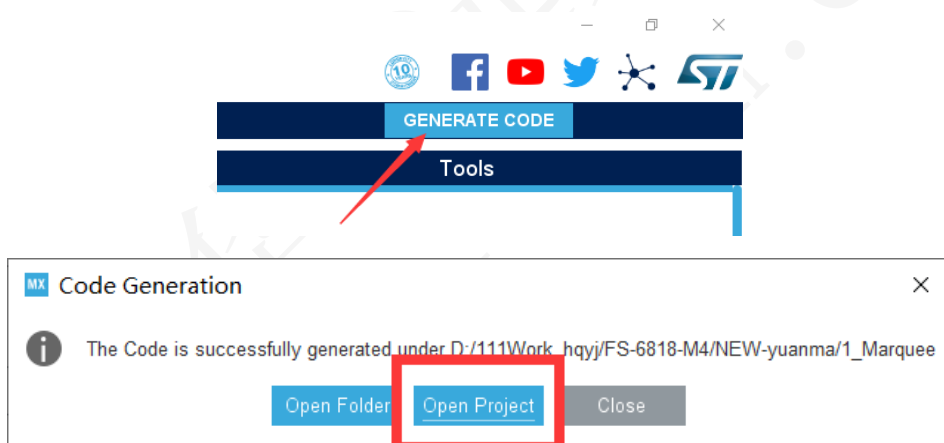


(7) 配置工程名字，工程路径和 IDE 软件（MDK-ARM V5.27），注意不要有中文路径。

(8) 工程软件生成配置，配置工程为 C 文件和 h 文件分开，以及拷贝使用到的 HAL 库文件，其他的文件不拷贝。



(9) 最后点击右上角的“GENWRITE CODE”，生成工程，生成工程完成后会弹出提示框，选择 Open Project 可直接从 MDK 中打开工程代码。



3.1.7 相关的 HAL 库函数

在生成的工程中，stm32f4xx_hal_gpio.c 文件中包含了 GPIO 相关的函数。依据函数名称可以推断出函数大致的功能，从而便于移植和阅读。这是 HAL 库函数命名的特点。下面详细介绍几个常用的函数。

1) GPIO 初始化函数 HAL_GPIO_Init

函数原型：void HAL_GPIO_Init(GPIO_TypeDef *GPIOx, GPIO_InitTypeDef *GPIO_Init)

函数作用：根据参数初始化 GPIOx/LPGPIOx 外设。

函数形参：

第一个形参：端口号，如 GPIOA、GPIOB 等。其类型 GPIO_TypeDef 是一个结构体，包含了该端口的相关寄存器的偏移地址。

```
typedef struct
{
    __IO uint32_t MODER; /*!< GPIO port mode register, Address offset: 0x00 */
    __IO uint32_t OTYPER; /*!< GPIO port output type register, Address offset: 0x04 */
    __IO uint32_t OSPEEDR; /*!< GPIO port output speed register, Address offset: 0x08 */
    __IO uint32_t PUPDR; /*!< GPIO port pull-up/pull-down register, Address offset: 0x0C */
    __IO uint32_t IDR; /*!< GPIO port input data register, Address offset: 0x10 */
    __IO uint32_t ODR; /*!< GPIO port output data register, Address offset: 0x14 */
    __IO uint32_t BSRR; /*!< GPIO port bit set/reset register, Address offset: 0x18 */
    __IO uint32_t LCKR; /*!< GPIO port configuration lock register, Address offset: 0x1C */
    __IO uint32_t AFR[2]; /*!< GPIO alternate function registers, Address offset: 0x20-0x24 */
} GPIO_TypeDef;
```

因此，其输入参数 GPIOx 应该是特定端口的寄存器起始地址（如 GPIOA 的寄存器起始地址）。在此起始地址后面，由 GPIO_TypeDef 结构体可知，第一个 32 位寄存器是 MODER，第二个是 OTYPER，以此类推。

第二个参数是一个类型为 GPIO_InitTypeDef 的结构体，该结构体包含下图所示几个成员。

```
typedef struct
{
    uint32_t Pin; /*!< Specifies the GPIO pins to be configured.
                  This parameter can be any value of @ref GPIO_pins_define */

    uint32_t Mode; /*!< Specifies the operating mode for the selected pins.
                   This parameter can be a value of @ref GPIO_mode_define */

    uint32_t Pull; /*!< Specifies the Pull-up or Pull-Down activation for the selected pins.
                   This parameter can be a value of @ref GPIO_pull_define */

    uint32_t Speed; /*!< Specifies the speed for the selected pins.
                    This parameter can be a value of @ref GPIO_speed_define */

    uint32_t Alternate; /*!< Peripheral to be connected to the selected pins.
                        This parameter can be a value of @ref GPIO_Alternate_function_selection */
}GPIO_InitTypeDef;
```

通过对这些成员赋值，再将配置好的结构体作为形参送入到函数中，即可实现对 GPIO 引脚的初始化操作。在 stm32f4xx_hal_gpio.h 文件中，已经定义好了每个成员的不同取值所代表的功能，下图所示为 Mode 成员所能配置的值。其他定义请参考该文件内相关程序。

```
/** @defgroup GPIO_mode_define GPIO mode define
 * @brief GPIO Configuration Mode
 * Elements values convention: 0xX0yz00YZ
 * - X : GPIO mode or EXTI Mode
 * - y : External IT or Event trigger detection
 * - z : IO configuration on External IT or Event
 * - Y : Output type (Push Pull or Open Drain)
 * - Z : IO Direction mode (Input, Output, Alternate or Analog)
 * @{
 */
#define GPIO_MODE_INPUT 0x00000000U /*!< Input Floating Mode */
#define GPIO_MODE_OUTPUT_PP 0x00000001U /*!< Output Push Pull Mode */
#define GPIO_MODE_OUTPUT_OD 0x00000011U /*!< Output Open Drain Mode */
#define GPIO_MODE_AF_PP 0x00000002U /*!< Alternate Function Push Pull Mode */
#define GPIO_MODE_AF_OD 0x00000012U /*!< Alternate Function Open Drain Mode */
#define GPIO_MODE_ANALOG 0x00000003U /*!< Analog Mode */
```

由 STM32CubeMX 生成的工程会在 MX_GPIO_Init 函数内调用 HAL_GPIO_Init 对引脚进行配置。

2) GPIO 读取引脚函数 HAL_GPIO_ReadPin

函数原型：GPIO_PinState HAL_GPIO_ReadPin(const GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin)

函数作用：读取对应的输入端口引脚上的值。

函数形参：

第一个形参：端口号，如 GPIOA、GPIOB 等。

第二个形参：引脚号，如 GPIO_PIN_0、GPIO_PIN_1 等。

返回值：对应引脚读到的值。

3) GPIO 写入引脚函数 HAL_GPIO_WritePin

函数原型：void HAL_GPIO_WritePin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin, GPIO_PinState PinState)

函数作用：设置对应引脚的电平高低。通过设置 BSRR 寄存器或 BRR 寄存器实现。

函数形参：

第一个形参：端口号，如 GPIOA、GPIOB 等。

第二个形参：引脚号，如 GPIO_PIN_0、GPIO_PIN_1 等。

第三个形参：引脚状态，可设置为 GPIO_PIN_RESET 或 GPIO_PIN_SET。

4) GPIO 翻转引脚函数 HAL_GPIO_TogglePin

函数原型：void HAL_GPIO_TogglePin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin)

函数作用：翻转对应引脚的电平。通过读取 ODR 寄存器和设置 BSRR 寄存器实现。

函数形参：

第一个形参：端口号，如 GPIOA、GPIOB 等。

第二个形参：引脚号，如 GPIO_PIN_0、GPIO_PIN_1 等。

3.1.8 编写代码

在打开的工程中打开 main.c 文件，找到 USER CODE BEGIN 4 位置，填写下面的代码。用户编写的代码都要写到 USER CODE BEGIN 和 USER CODE END 之间，避免 STM32CubeMX 再次生成代码时覆盖掉。

```
/* USER CODE BEGIN 4 */
void Turn_On_LED(uint8_t LED_NUM)
{
    switch(LED_NUM)
    {
        case 0:
            HAL_GPIO_WritePin(GPIOH,GPIO_PIN_15,GPIO_PIN_RESET); /*点亮 D4 灯*/
            break;
        case 1:
            HAL_GPIO_WritePin(GPIOB,GPIO_PIN_15,GPIO_PIN_RESET); /*点亮 D3 灯*/
            break;
        case 2:
            HAL_GPIO_WritePin(GPIOC,GPIO_PIN_0,GPIO_PIN_RESET); /*点亮 D2 灯*/
            break;
        case 3:
            HAL_GPIO_WritePin(GPIOF,GPIO_PIN_10,GPIO_PIN_RESET); /*点亮 D1 灯*/
            break;
        default:
            break;
    }
}
/* USER CODE END 4 */
```

在文件前部声明该函数。

```
/* USER CODE BEGIN PFP */
void Turn_On_LED(uint8_t LED_NUM);
/* USER CODE END PFP */
```

在 main 函数中定义变量用于计数。

```
/* USER CODE BEGIN 1 */
uint8_t count=0;
/* USER CODE END 1 */
```

在主循环中添加下面的代码。

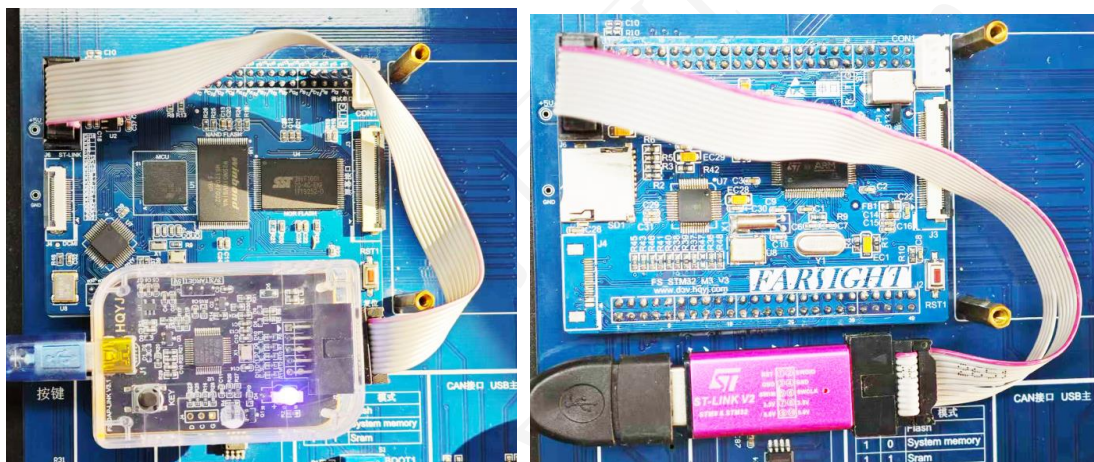
```
/* USER CODE BEGIN WHILE */
while (1)
```

```
{
    //初始化四个 LED 的状态，灭
    HAL_GPIO_WritePin(GPIOB,GPIO_PIN_15,GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOF,GPIO_PIN_10,GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOH,GPIO_PIN_15,GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOC,GPIO_PIN_0,GPIO_PIN_SET);
    Turn_On_LED(count%4);    //点亮一个 LED 灯
    count++;
    HAL_Delay(500);
/* USER CODE END WHILE */
```

编写完成后，单击 Build 按钮进行编译。



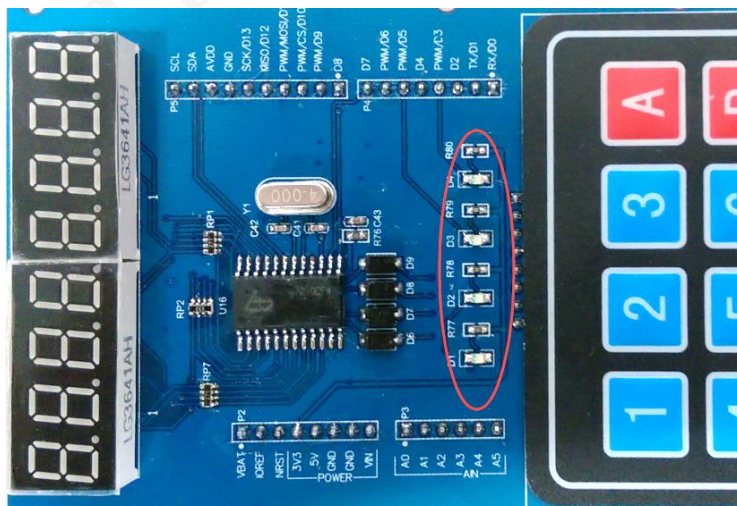
若编译没有错误，则单击 Download 按钮将其下载到开发板里。注意开发板要提前上电。ST-Link 和实验箱的连接方式如下图所示。



下载时若报错，可尝试重新插拔 ST-Link 或者按下 ST-Link 上的复位键。

3.1.9 实验现象

可以看到按键数码管扩展板上的 LED 灯 D1、D2、D3、D4 循环亮灭。



3.2 蜂鸣器实验

3.2.1 实验目的

蜂鸣器是一种一体化结构的电子讯响器，采用直流电压供电，广泛应用于计算机、打印机、报警器、电子玩具、汽车电子设备、电话机、定时器等电子产品中作发声器件，用于产品的声音提醒或者警告。

通过该实验，学习者将学会如何使用 STM32 的 GPIO 引脚来控制蜂鸣器的开关状态。实验中，GPIO 引脚配置为输出模式，并通过编程控制输出高电平或低电平，进而控制蜂鸣器的工作状态（如响或不响）。

通过实际控制蜂鸣器，学习者可以更好地理解蜂鸣器的工作原理。蜂鸣器通常有两种类型：有源蜂鸣器和无源蜂鸣器。通过实验，学习者可以明确如何通过不同的信号来驱动这两种蜂鸣器。

3.2.2 实验内容

编写蜂鸣器程序，实现让传感器扩展板上的蜂鸣器按一定频率发出响声。调试编译好程序后，将程序下载到开发板，按复位键观察实验现象。

本实验例程：【实验代码/2_Beep】。

3.2.3 蜂鸣器

3.2.3.1 传感器介绍

蜂鸣器是一种能够产生声音的电子元件，常用于警报、提醒和提示等应用中。它可以发出不同频率和持续时间的声音信号，用于吸引注意、传达信息或产生警报。

蜂鸣器可以根据其工作原理和结构特点进行分类。以下是几种常见的蜂鸣器分类：

传统电磁式蜂鸣器是最常见的一种类型。它由一个线圈和一个振动片组成。当通过线圈通电时，产生的磁场会使振动片振动，从而产生声音。这种蜂鸣器通常需要外部电路提供驱动信号。



压电式蜂鸣器使用压电材料（如陶瓷）的压电效应来产生声音。当施加电压或电场时，压电材料会发生形变，从而产生声波。压电式蜂鸣器具有体积小、功耗低和响应速度快等优点，常用于电子设备中。

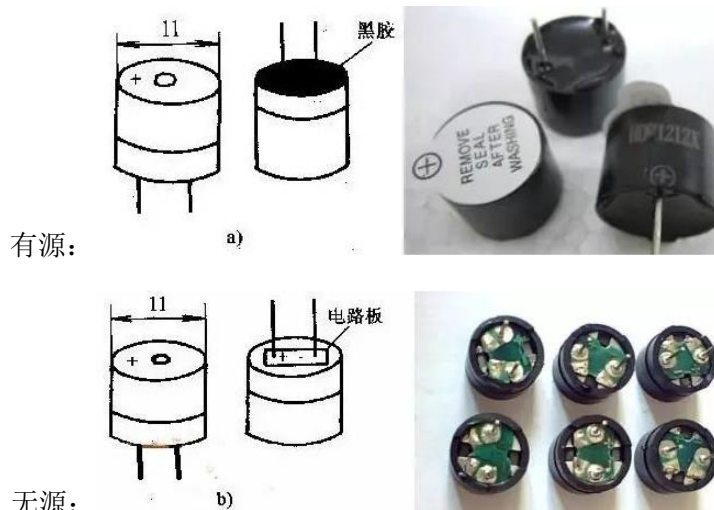


电动式蜂鸣器由电机和振动片组成。当电机转动时，振动片会产生声音。这种蜂鸣器通常需要外部电源提供驱动。



MEMS 蜂鸣器采用微机电系统（MEMS）技术制造，结构小巧。它利用微小的机械振动来产生声音，通常具有低功耗和高可靠性。

根据内部有无震荡源，蜂鸣器还分为两种类型：有源蜂鸣器和无源蜂鸣器。



有源:

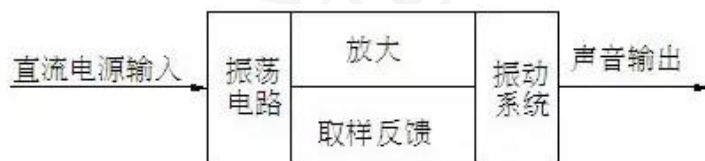
无源:

3.2.3.2 应用场景

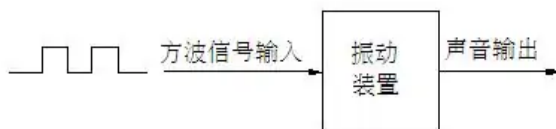
计算机行业（主板蜂鸣器，机箱蜂鸣器，电脑蜂鸣器）打印机（控制板蜂鸣器）、复印机、报警器行业（报警蜂鸣器，警报蜂鸣器）、电子玩具（音乐蜂鸣器）、农业、汽车电子设备行业（车载蜂鸣器，倒车蜂鸣器，汽车蜂鸣器，摩托车蜂鸣器）电话机（环保蜂鸣器）、定时器，空调，医疗设备等电声行业。

3.2.3.3 原理及特性

有源自激型蜂鸣器的工作发声原理是：直流电源输入经过振荡系统的放大取样电路在谐振装置作用下产生声音信号，有源自激型蜂鸣器的工作发声原理图如图：

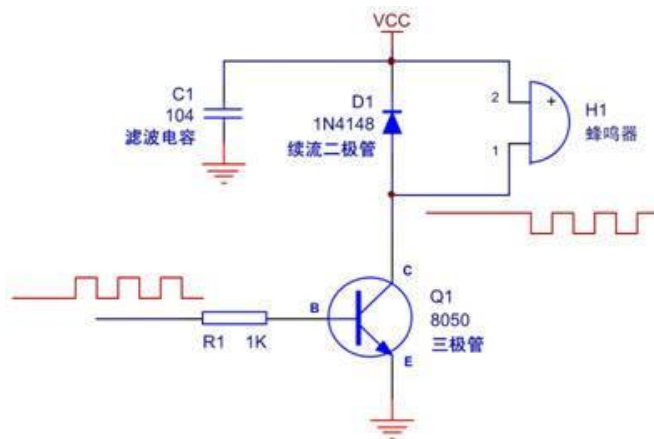


无源他激型蜂鸣器的工作发声原理是：方波信号输入谐振装置转换为声音信号输出，无源他激型蜂鸣器的工作发声原理图如图：



由于蜂鸣器的工作电流一般比较大，以致于单片机的 I/O 口是无法直接驱动的（但 AVR 可以驱动小功率蜂鸣器），所以要利用放大电路来驱动，一般使用三极管来放大电流就可以了。

蜂鸣器驱动电路一般都包含以下几个部分：一个三极管、一个蜂鸣器、一个续流二极管和一个电源滤波电容。



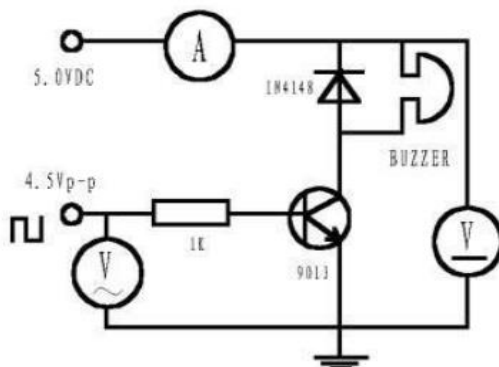
蜂鸣器：发声元件，在其两端施加直流电压（有源蜂鸣器）或者方波（无源蜂鸣器）就可以发声，其主要参数是外形尺寸、发声方向、工作电压、工作频率、工作电流、驱动方式（直流/方波）等。这些都可以根据需要来选择。

续流二极管：蜂鸣器本质上是一个感性元件，其电流不能瞬变，因此必须有一个续流二极管提供续流。否则，在蜂鸣器两端会产生几十伏的尖峰电压，可能损坏驱动三极管，并干扰整个电路系统的其它部分。

滤波电容：滤波电容 C1 的作用是滤波，滤除蜂鸣器电流对其它部分的影响，也可改善电源的交流阻抗，如果可能，最好是再并联一个 220uF 的电解电容。

三极管：三极管 Q1 起开关作用，其基极的高电平使三极管饱和导通，使蜂鸣器发声；而基极低电平则使三极管关闭，蜂鸣器停止发声。

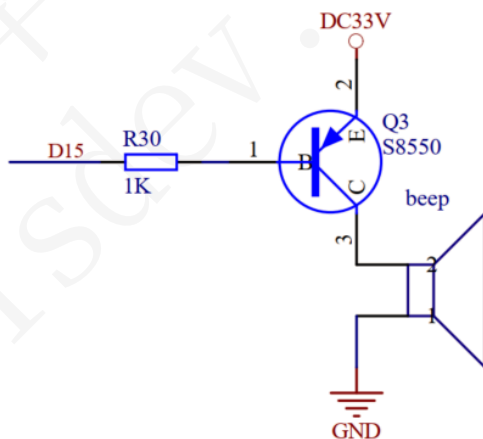
3.2.3.4 数据格式及接口



对于无源蜂鸣器来说，根据原理，我们需要将左侧的输入端口连接到单片机的 GPIO 引脚上，并让 GPIO 引脚发出一定频率的 PWM 波，即可使能蜂鸣器。

3.2.4 硬件原理

打开传感器板原理图.pdf，找到蜂鸣器电路。

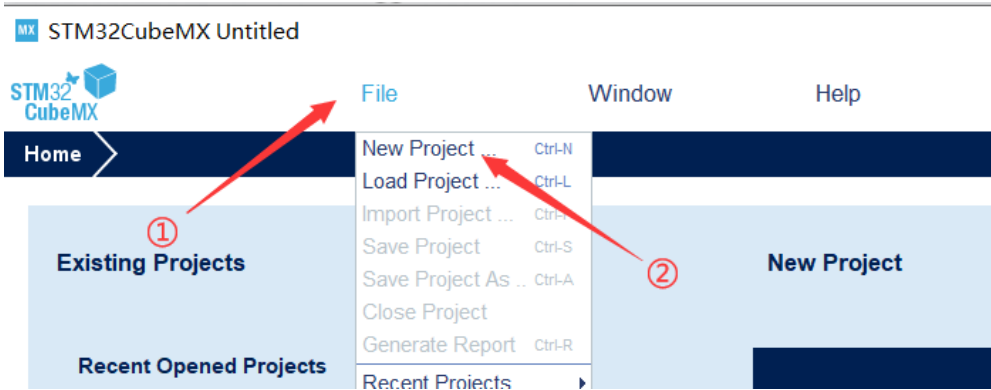


由于 STM32 的引脚驱动电流有限，且在驱动感性器件时直接连接容易损坏芯片，所以原理图上通过三极管来进行电流放大，从而驱动蜂鸣器。Q3 为 PNP 型三极管，当 D15 上为低电平时三极管导通。

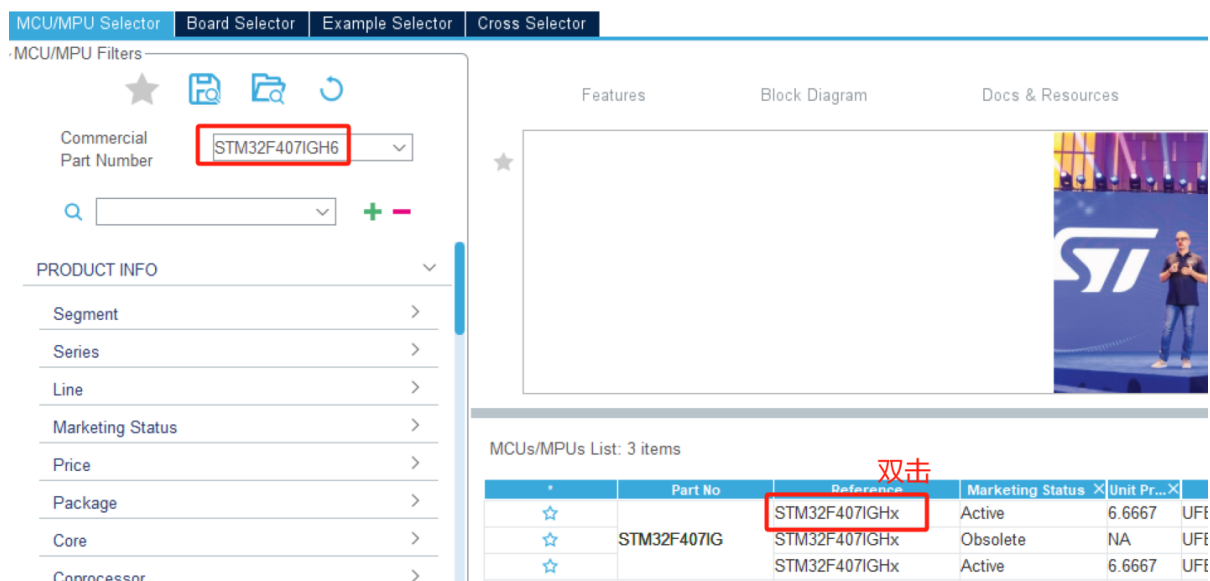
按照 3.1.5 节的方法即可找到 D15 引脚连接到了 STM32F407 的 PG6 引脚上。

3.2.5 STM32CubeMX 配置

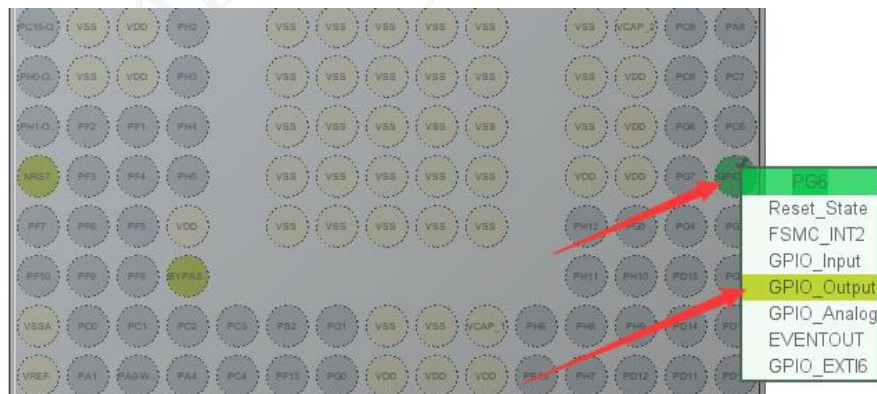
1. 打开 STM32CubeMX->点击左上角的 File->点击 New Project



2. 选择芯片，查看芯片的型号，在搜索框输入芯片型号，本实验中选用的芯片是 STM32F407IGHx

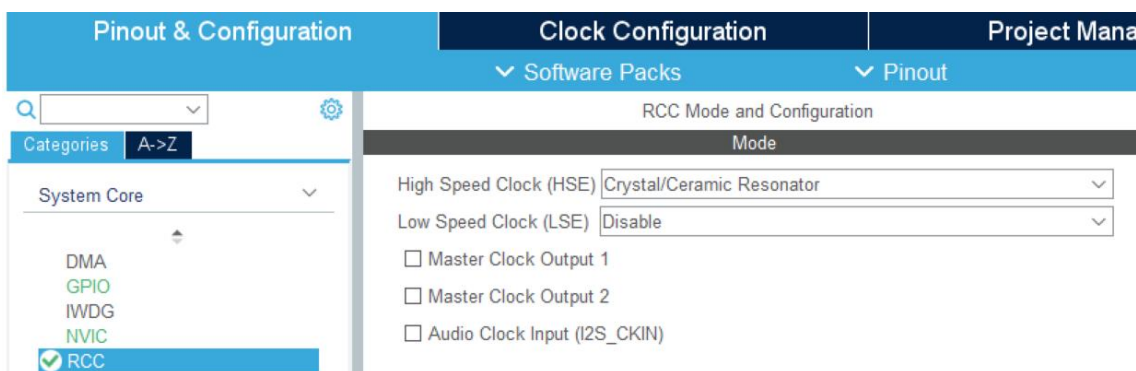


3. 对管脚进行配置。找到 PG6，将管脚设为 GPIO_Output 模式。

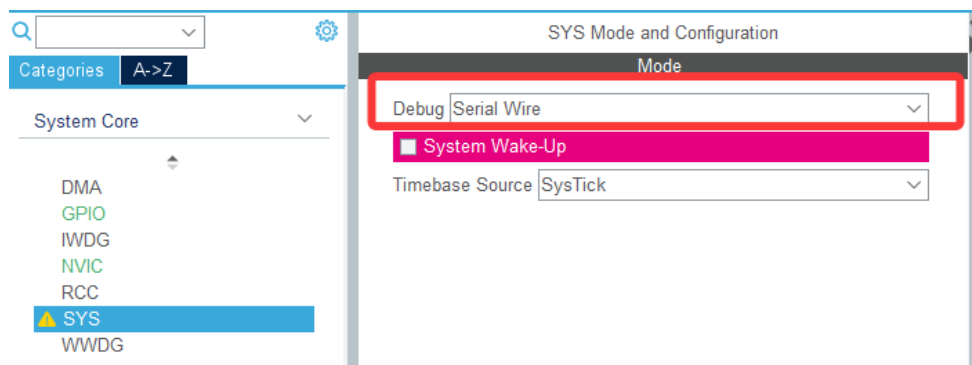


配置完后再 GPIO 功能页即可看到这个引脚的配置情况。为 Output Push Pull（推挽输出）模式，默认电平为低电平，无上下拉电阻，输出速度为低。

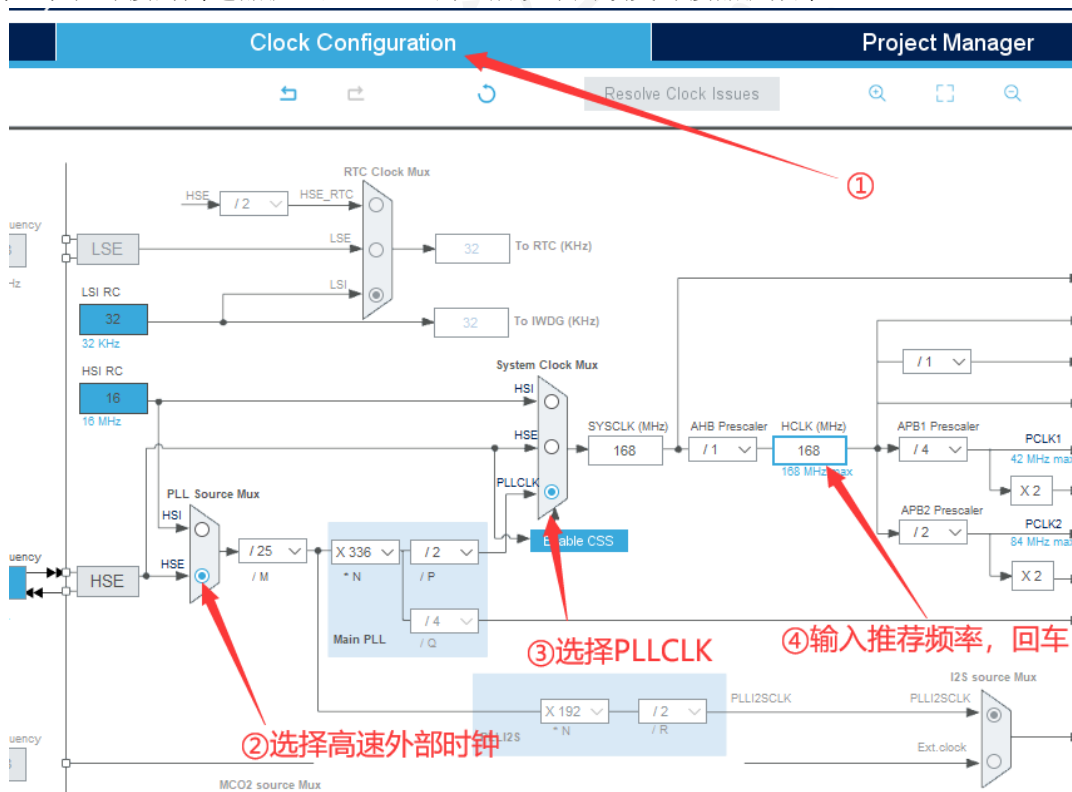
4. 配置 RCC，点击左侧的 RCC，找到 High Speed Clock (HSE)，点击选择 Crystal/Ceramic Resonator。



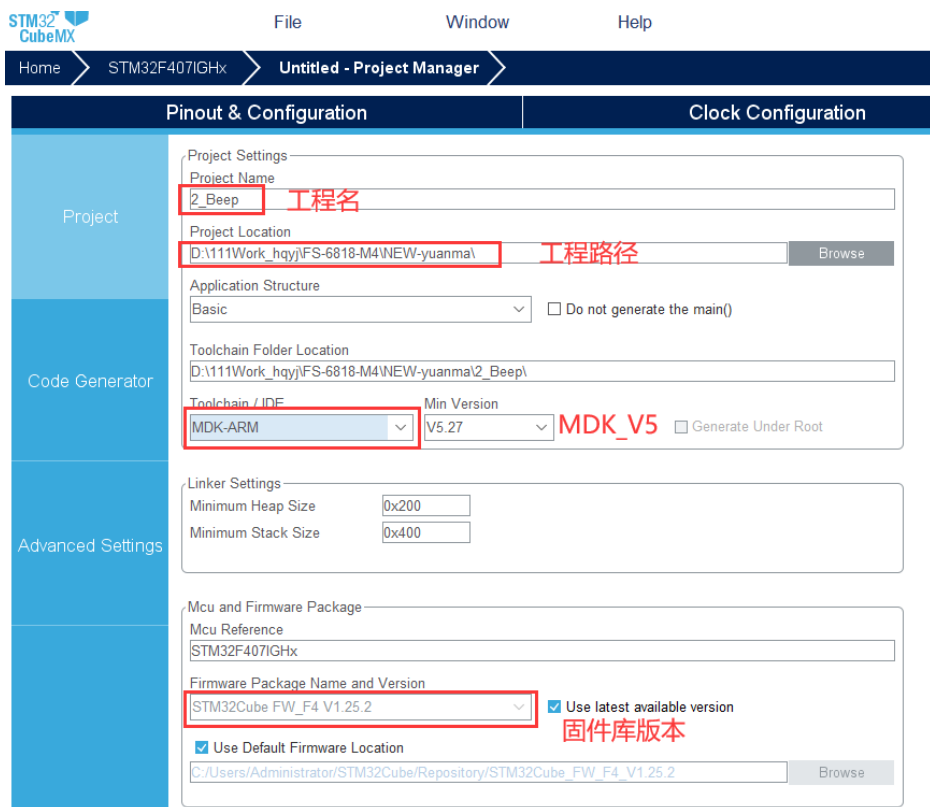
5. 将调试模式修改为 SWD 模式:



6. 点击上方的 Clock Configuration, 按下图所示配置时钟树。因为核心板上外接的晶振是 25MHz 的, 时钟树上默认外接的高速晶振也是 25MHz 的, 所以不用改变外接晶振频率。



7. 配置工程名字, 工程路径和 IDE 软件 (MDK-ARM V5.27), 注意不要有中文路径。



8. 工程软件生成配置，配置工程为 C 文件和 h 文件分开，以及拷贝使用到的 HAL 库文件，其他的文件不拷贝，最后点击右上角的“GENWRTE CODE”，生成工程。



9. 生成工程完成后会弹出提示框，选择 Open Project 可直接从 MDK 中打开工程代码。

3.2.6 编写代码

在主循环中加入如下代码：

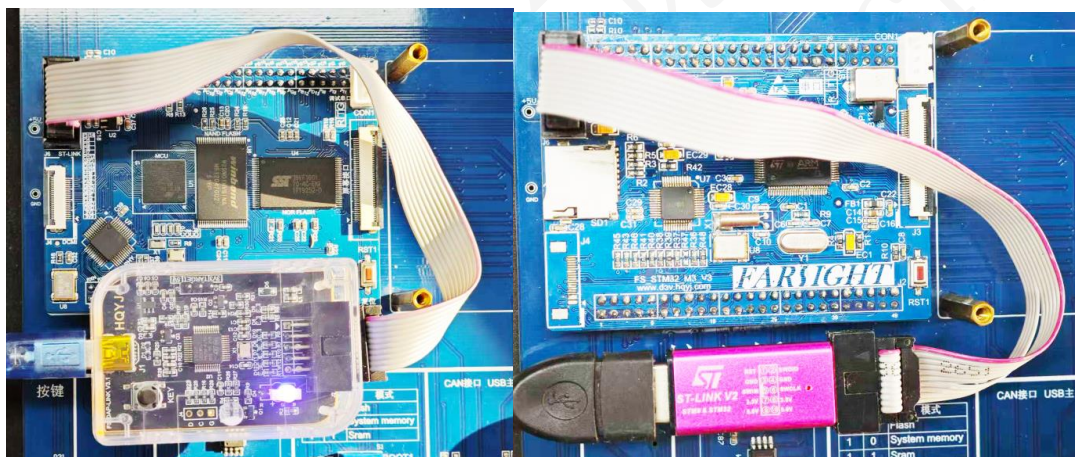
```
/* USER CODE BEGIN WHILE */
while (1)
{
    HAL_GPIO_WritePin(GPIOG,GPIO_PIN_6,GPIO_PIN_SET);//高电平
    HAL_Delay(2);//延时
    HAL_GPIO_WritePin(GPIOG,GPIO_PIN_6,GPIO_PIN_RESET);//低电平
    HAL_Delay(2);//延时
}
/* USER CODE END WHILE */
```

通过改变延时的时间即可改变蜂鸣器振动的频率。

编写完成后，单击 Build 按钮进行编译。



若编译没有错误，则单击 Download 按钮将其下载到开发板里。注意开发板要提前上电。ST-Link 和实验箱的连接方式如下图所示。



下载时若报错，可尝试重新插拔 ST-Link 或者按下 ST-Link 上的复位键。

3.2.7 实验现象

可以听到蜂鸣器发出一定频率的声音。

3.3 舵机实验

3.3.1 实验目的

通过实验，学习者将了解舵机的工作机制。舵机通过接收不同的 PWM 信号来改变其旋转角度，通常是通过调整信号的占空比来实现这一目的。学习者将了解舵机在不同占空比下的响应，并通过实验观察舵机如何根据 PWM 信号调整角度。

在本实验中，学习者需要通过 STM32 的定时器模块来生成 PWM 波形。实验有助于学习者理解如何配置 STM32 的定时器、如何设定 PWM 波的频率、如何调整占空比，以控制舵机的旋转角度。

3.3.2 实验内容

首先将 TIM12 的通道 1 设置成 PWM 模式，然后在主循环中不断改变 PWM 输出信号的占空比，即可控制舵机的旋转角度。

本实验例程：【实验代码/3_Steering_Engine】。

3.3.3 PWM 简介

PWM (Pulse Width Modulation, 脉冲宽度调制) 是一种通过调节信号的占空比来控制输出的技术。PWM 广泛应用于电机控制、亮度调节、音频信号处理等多个领域，尤其在嵌入式系统中，它是实现高效、精确控制的重要工具。

PWM 信号是一种数字信号，它的特点是周期性地切换高电平（逻辑 1）和低电平（逻辑 0），通过改变高电平持续的时间来调节信号的占空比（Duty Cycle）。其基本参数包括：

频率 (Frequency)： 信号每秒钟周期的次数（单位：Hz），即 PWM 波形的重复频率。

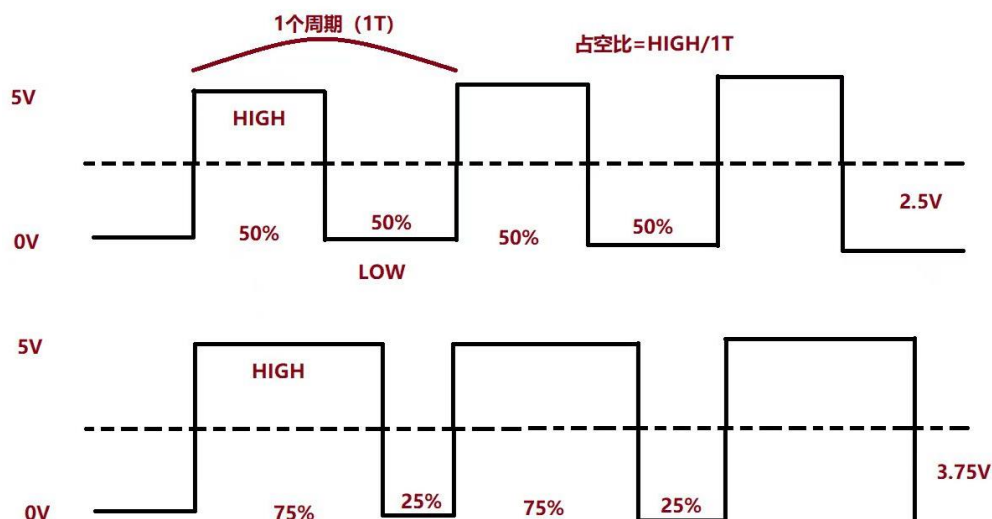
占空比 (Duty Cycle)： PWM 信号中高电平持续的时间占总周期的比例，通常表示为百分比。例如，占空比为 50% 意味着信号有一半的时间为高电平，另一半为低电平。

占空比 = 高电平持续时间 / 总周期时间 × 100%

比方说一个 PWM 信号的周期是 10ms，脉宽时间是 8ms，占空比就是 80%。

PWM 信号的波形：

PWM 信号的波形可以用一个简单的矩形波表示。如下图所示：



占空比决定了信号的平均功率，因此通过改变占空比可以控制输出设备的响应效果。

PWM 的应用：

电机控制： 在电机控制中，PWM 被用于调节电机的速度。通过调节 PWM 信号的占空比，能够控制电机所接收到的功率，从而控制电机的转速。例如，使用 PWM 控制直流电机时，占空比越大，电机转速越快。

亮度调节： 在 LED 照明中，PWM 用于调节 LED 的亮度。通过调节 LED 驱动电流的占空比，可以实现从完全关闭到最大亮度的调节。这种方法比通过改变电流强度来调节亮度更节能，因为 PWM 只是快速地开启和关闭 LED，而不是持续地调整电流。

音频信号生成： PWM 还广泛应用于音频信号的生成和处理。在一些音频系统中，可以通过 PWM 信号来模拟模拟信号，或者用于控制音量。

电源控制： 在开关电源（如 DC-DC 转换器）中，PWM 信号用于控制开关的开闭周期，从而调节输出电压或电流。

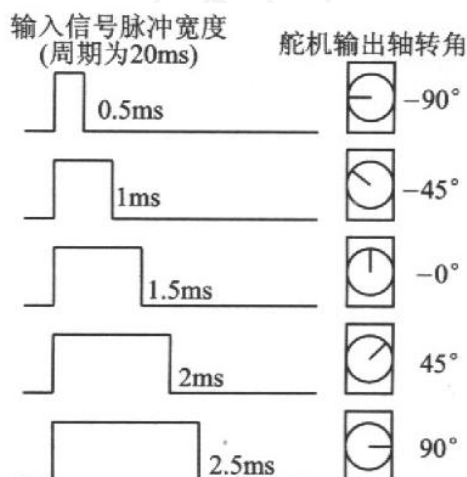
3.3.4 舵机简介

舵机（Servo Motor）是一种能够精确控制角度的电动机，通常用于需要精确定位和控制的应用场景。舵机的特点是能够根据输入的信号（通常是 PWM 信号）来调整其角度，常见的舵机有许多应用，尤其在机器人、遥控模型、自动化设备等领域广泛使用。



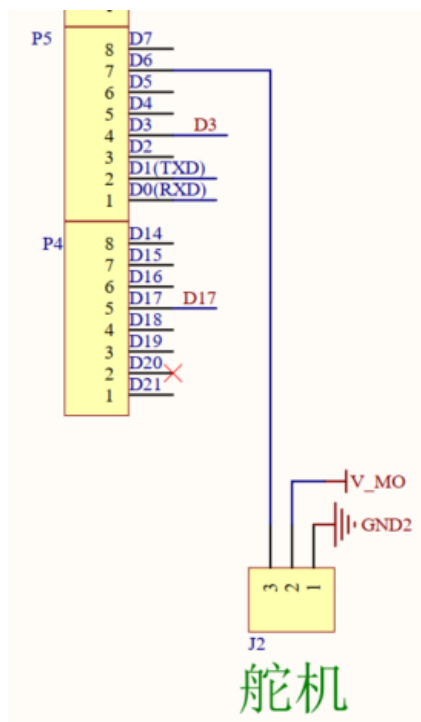
舵机的工作原理：

舵机通过接收控制信号（通常是 PWM 脉宽调制信号）来控制其转动的角度。PWM 信号的占空比（即信号高电平的持续时间）决定了舵机的旋转角度。舵机的控制一般需要一个 20ms 左右的时基脉冲，该脉冲的高电平部分一般为 0.5ms-2.5ms 范围内的角度控制脉冲部分，总间隔为 2ms。以 180 度角度伺服为例，那么对应的控制关系是这样的：



3.3.5 硬件原理

打开电机板原理图.pdf。可以看到舵机的控制引脚连接到了 P5 座上的 D6 引脚。



按照 3.1.5 节的方法即可找到 D6 引脚连接到了 STM32F407 的 PB14 引脚上。该引脚可被功能复用为 TIM12 的通道 1。

3.3.6 STM32 的定时器

定时器用于生成精确的时间延迟、周期性触发事件以及测量时间间隔等任务。它们通常用于实现各种定时和计数功能，例如控制脉冲的频率、PWM（脉冲宽度调制）、输入捕获和输出比较等。STM32 的定时器通常包括基本定时器、通用定时器和高级定时器等不同类型。它们具有不同的功能和特性，可以根据应用的需求选择合适的类型。

STM32F4 单片机共有 14 个定时器，分别是 2 个高级定时器（TIM1、TIM8）、10 个通用定时器（TIM2 到 TIM5、TIM9 到 TIM14）和 2 个基本定时器（TIM6、TIM7）。

TIM9 到 TIM14 通用定时器包含一个 16 位自动重载计数器，该计数器由可编程预分频器驱动。它们可用于多种用途，包括测量输入信号的脉冲宽度（输入捕获），或者生成输出波形（输出比较、PWM）。使用定时器预分频器和 RCC 时钟控制器预分频器，可将脉冲宽度和波形周期从几微秒调制到几毫秒。TIM9 到 TIM14 定时器彼此完全独立，不共享任何资源。

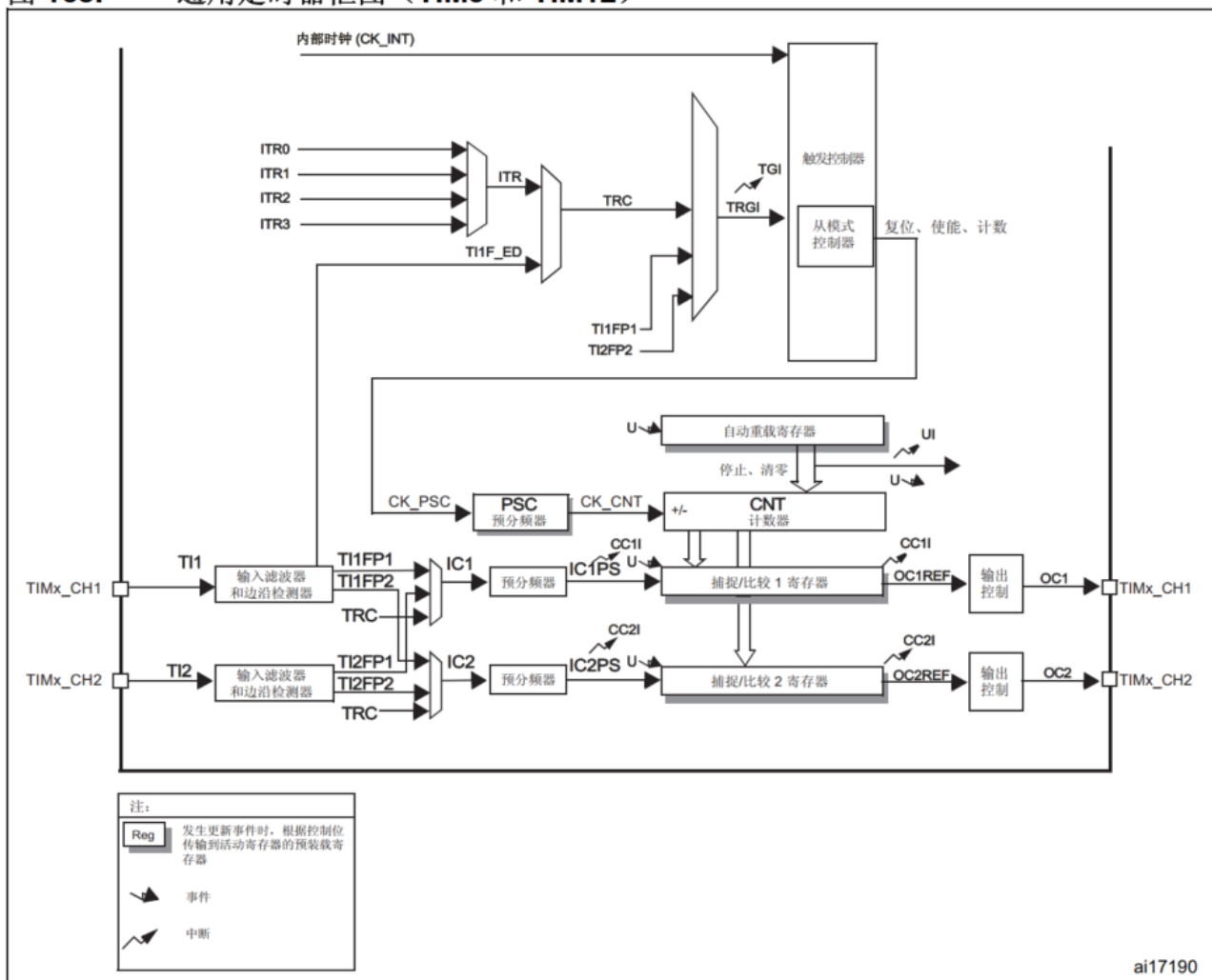
TIM9 到 TIM14 通用定时器具有以下特性：

- 16 位自动重载递增计数器（属于中等容量器件）
- 16 位可编程预分频器，用于对计数器时钟频率进行分频（即运行时修改），分频系数介于 1 和 65536 之间
- 多达 2 个独立通道，可用于：
 - 输入捕获
 - 输出比较
 - PWM 生成（边沿对齐模式）
 - 单脉冲模式输出
- 使用外部信号控制定时器且可实现多个定时器互连的同步电路
- 发生如下事件时生成中断：
 - 更新：计数器上溢、计数器初始化（通过软件或内部触发）
 - 触发事件（计数器启动、停止、初始化或者由内部触发计数）
 - 输入捕获

一 输出比较

下图为通用定时器 TIM12 的内部原理框图。

图 165. 通用定时器框图 (TIM9 和 TIM12)



3.3.7 定时器的 PWM 模式

在 PWM 模式下，PWM 的频率由 TIMx_ARR 寄存器的值确定，占空比由 TIMx_CCRx 寄存器的值确定。

通过向 TIMx_CCMRx 寄存器中的 OCxM 位写入 110 (PWM 模式 1) 或 111 (PWM 模式 2)，可以独立选择各通道 (每个 OCx 输出对应一个 PWM) 的 PWM 模式。必须通过将 TIMx_CCMRx 寄存器中的 OCxPE 位置 1 使能相应预装载寄存器，最后通过将 TIMx_CR1 寄存器中的 ARPE 位置 1 使能自动重载预装载寄存器 (在递增计数或中心对齐模式下)。

由于只有在发生更新事件时预装载寄存器才会传送到影子寄存器，因此启动计数器之前，必须通过将 TIMx_EGR 寄存器中的 UG 位置 1 来初始化所有寄存器。

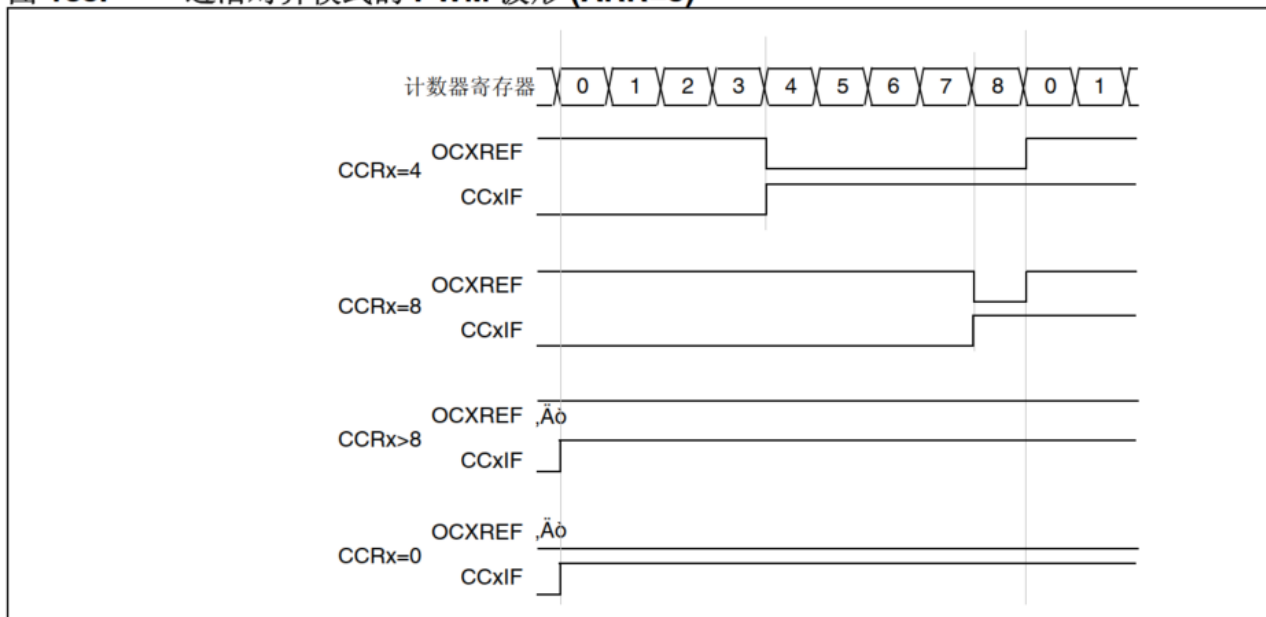
OCx 极性可使用 TIMx_CCER 寄存器的 CCxP 位来编程。既可以设为高电平有效，也可以设为低电平有效。OCx 输出通过将 TIMx_CCER 寄存器中的 CCxE 位置 1 来使能。有关详细信息，请参见 TIMx_CCERx 寄存器说明。

在 PWM 模式 (1 或 2) 下，TIMx_CNT 总是与 TIMx_CCRx 进行比较，以确定是 $TIMx_CNT \leq TIMx_CCRx$ 。因为计数器采用递增方式计数，所以定时器能够在边沿对齐模式下生成 PWM。

PWM 边沿对齐模式

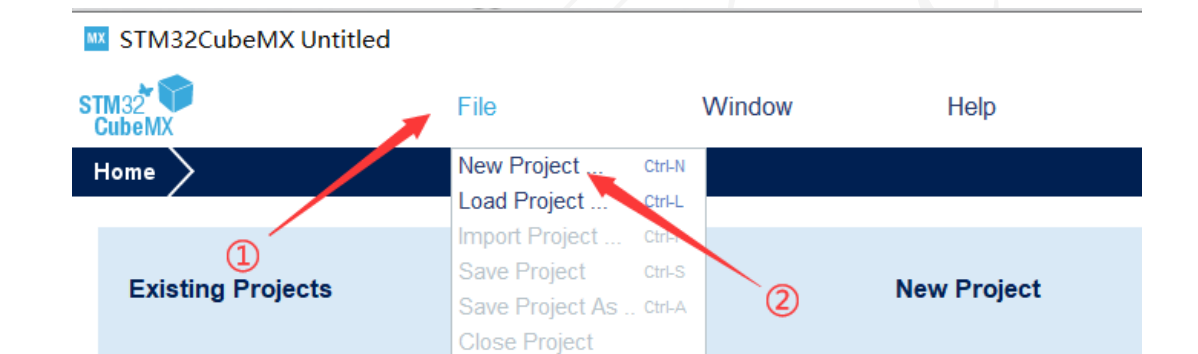
以下以 PWM 模式 1 为例。只要 $TIMx_CNT < TIMx_CCRx$ ，PWM 参考信号 OCxREF 便为高电平，否则为低电平。如果 TIMx_CCRx 中的比较值大于自动重载值 (TIMx_ARR 中)，则 OCxREF 保持为“1”。如果比较值为 0，则 OCxRef 保持为“0”。下图举例介绍边沿对齐模式的一些 PWM 波形 (TIMx_ARR=8)。

图 183. 边沿对齐模式的 PWM 波形 (ARR=8)

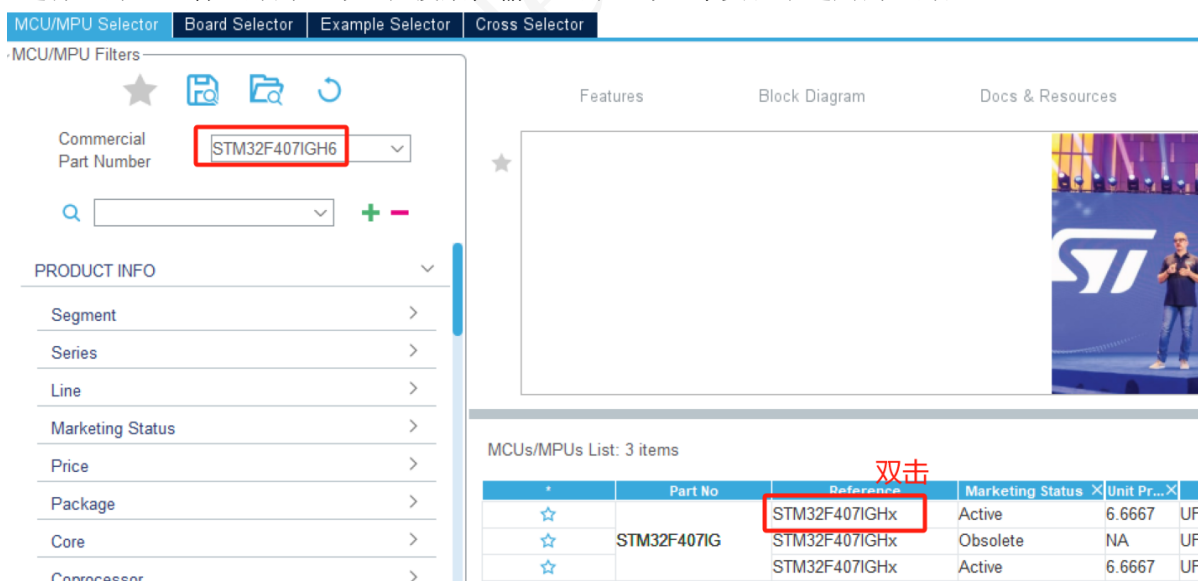


3.3.8 STM32CubeMX 配置

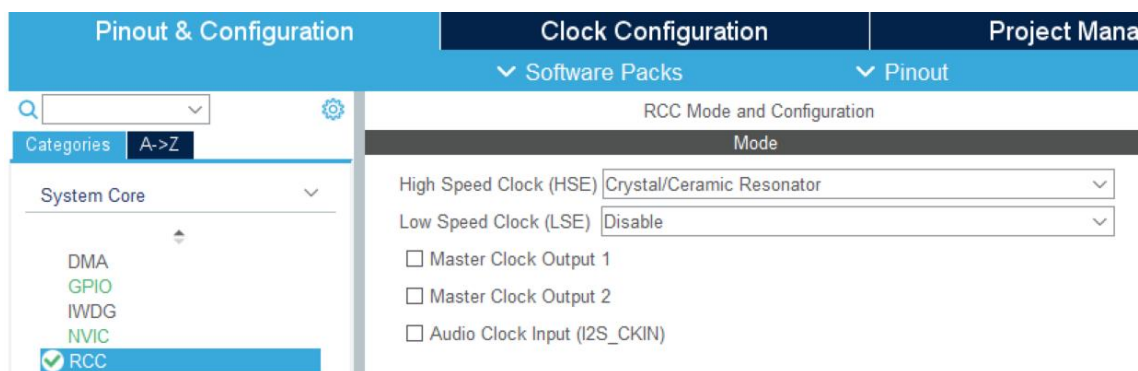
打开 STM32CubeMX→点击左上角的 File→点击 New Project。



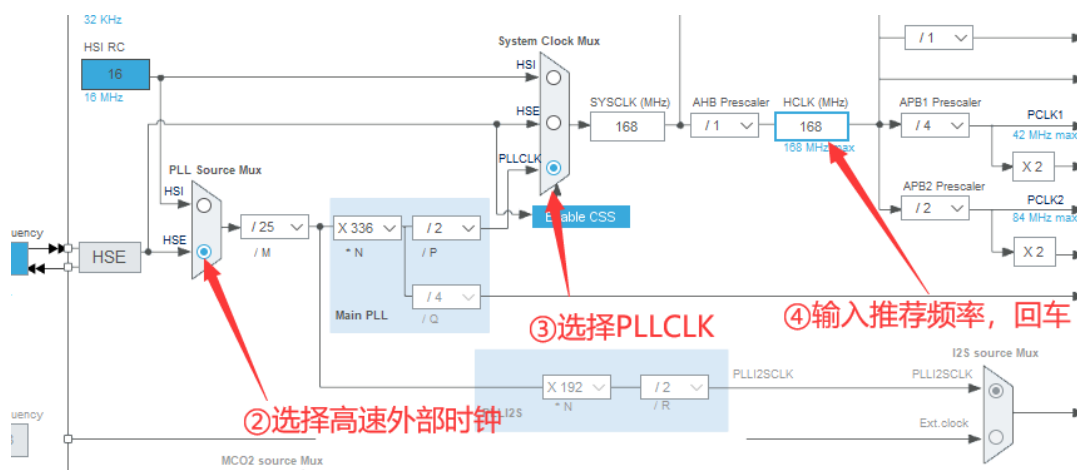
选择芯片，查看芯片的型号，在搜索框输入芯片型号，本实验中选用的芯片是 STM32F407IGHx



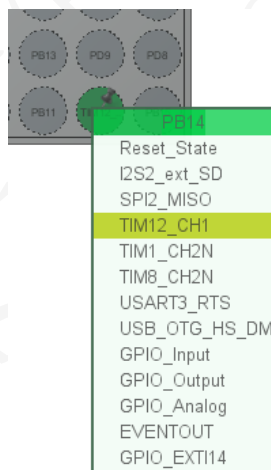
配置 RCC，点击左侧的 RCC，找到 High Speed Clock (HSE)，点击选择 Crystal/Ceramic Resonator。



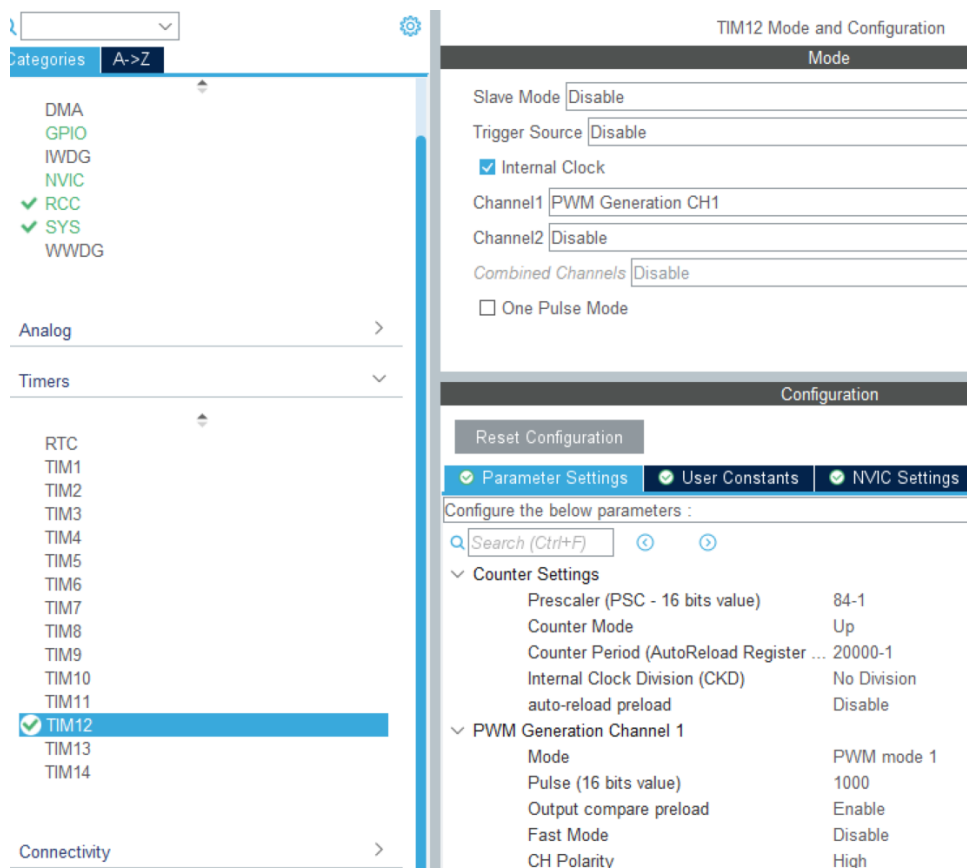
点击上方的 Clock Configuration，按下图所示配置时钟树。因为核心板上外接的晶振是 25MHz 的，时钟树上默认外接的高速晶振也是 25MHz 的，所以不用改变外接晶振频率。



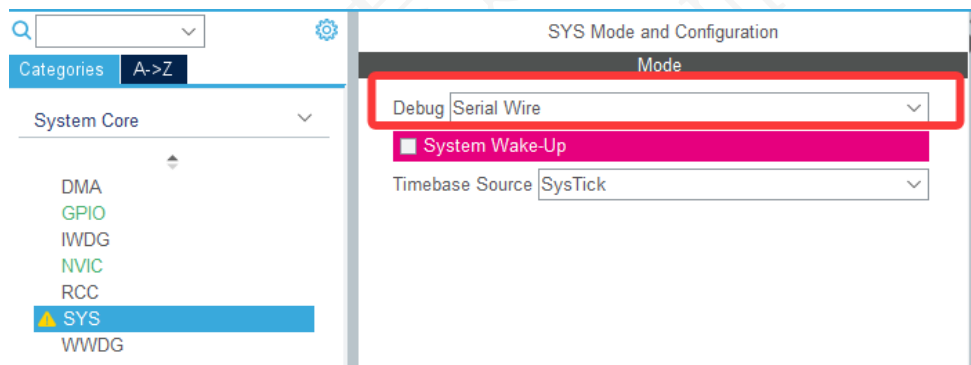
配置 PB14 引脚为 TIM12_CH1 模式



对 TIM12 进行初始化配置。TIM12 的输入时钟频率是 84MHz，此处分频 84 倍即可使得计数频率为 1MHz。注意分频倍数为 PSC+1。ARR 寄存器设置为 20000-1 即可在计数 20000 个后触发更新事件，即 20ms。同样意味着 PWM 周期是 20ms。在 Pulse 处写 1000，代表高电平为 1ms，即让舵机旋转 45°。



将调试模式修改为 SWD 模式：



配置工程名字，工程路径和 IDE 软件（MDK-ARM V5.27），（注意不要有中文路径）。

工程软件生成配置，配置工程为 C 文件和 h 文件分开，以及拷贝使用到的 HAL 库文件，其他的文件不拷贝，最后点击右上角的“GENWRITE CODE”，生成工程。

生成工程完成后会弹出提示框，选择 Open Project 可直接从 MDK 中打开工程代码。

3.3.9 相关的 HAL 库函数

3.3.10 PWM 的 HAL 库函数

HAL 库的 PWM 相关函数可以实现的功能有：

- 初始化和配置 TIM PWM。
- 取消初始化 TIM PWM。
- 启动 TIM PWM。
- 停止 TIM PWM。
- 启动 TIM PWM 并启用中断。

- 停止 TIM PWM 并禁用中断。
- 启动 TIM PWM 并启用 DMA 传输。
- 停止 TIM PWM 并禁用 DMA 传输。

下面介绍几个常用的函数：

1) PWM 初始化函数 HAL_TIM_PWM_Init

函数原型：HAL_StatusTypeDef HAL_TIM_PWM_Init(TIM_HandleTypeDef *htim)

函数作用：根据 TIM_HandleTypeDef 中指定的参数初始化 TIM 的 PWM 功能，并初始化相关的句柄。

函数形参：htim：指向包含对应 TIM 定时器配置信息的 TIM_HandleTypeDef 结构体。

返回值：HAL 状态。

2) PWM 开始函数 HAL_TIM_PWM_Start

函数原型：HAL_StatusTypeDef HAL_TIM_PWM_Start(TIM_HandleTypeDef *htim, uint32_t Channel)

函数作用：开始 PWM 信号生成。

函数形参：

第一个参数：指向包含对应 TIM 定时器配置信息的 TIM_HandleTypeDef 结构体。

第二个参数：选择通道。

返回值：HAL 状态。

3) PWM 停止函数 HAL_TIM_PWM_Stop

函数原型：HAL_StatusTypeDef HAL_TIM_PWM_Stop(TIM_HandleTypeDef *htim, uint32_t Channel)

函数作用：停止 PWM 信号生成。

函数形参：

第一个参数：指向包含对应 TIM 定时器配置信息的 TIM_HandleTypeDef 结构体。

第二个参数：选择通道。

返回值：HAL 状态。

4) 中断模式下的 PWM 开始函数 HAL_TIM_PWM_Start_IT

函数原型：HAL_StatusTypeDef HAL_TIM_PWM_Start_IT(TIM_HandleTypeDef *htim, uint32_t

Channel)

函数作用：开始带中断模式的 PWM 信号生成。

函数形参：

第一个参数：指向包含对应 TIM 定时器配置信息的 TIM_HandleTypeDef 结构体。

第二个参数：选择通道。

返回值：HAL 状态。

5) 中断模式下的 PWM 停止函数 HAL_TIM_PWM_Stop_IT

函数原型：HAL_StatusTypeDef HAL_TIM_PWM_Stop_IT(TIM_HandleTypeDef *htim, uint32_t Channel)

函数作用：停止带中断模式的 PWM 信号生成。

函数形参：

第一个参数：指向包含对应 TIM 定时器配置信息的 TIM_HandleTypeDef 结构体。

第二个参数：选择通道。

返回值：HAL 状态。

6) DMA 模式下的 PWM 开始函数 HAL_TIM_PWM_Start_DMA

函数原型：HAL_StatusTypeDef HAL_TIM_PWM_Start_DMA(TIM_HandleTypeDef *htim, uint32_t

Channel, const uint32_t *pData, uint16_t Length)

函数作用：用 DMA 模式开始 PWM 信号生成。

函数形参：

第一个参数：指向包含对应 TIM 定时器配置信息的 TIM_HandleTypeDef 结构体。

第二个参数：选择通道。

第三个参数：源缓存地址。

第四个参数：从内存传递给 TIM 外设的数据长度。

返回值：HAL 状态。

7) DMA 模式下的 PWM 停止函数 HAL_TIM_PWM_Stop_DMA

函数原型：HAL_StatusTypeDef HAL_TIM_PWM_Stop_DMA(TIM_HandleTypeDef *htim, uint32_t Channel)

函数作用：停止 DMA 模式的 PWM 信号生成。

函数形参：

第一个参数：指向包含对应 TIM 定时器配置信息的 TIM_HandleTypeDef 结构体。

第二个参数：选择通道。

返回值：HAL 状态。

3.3.11 编写代码

在 main() 函数中填写下列代码。

```
/* USER CODE BEGIN 2 */
// 启动定时器 12 的 PWM 输出信号，通道 1
HAL_TIM_PWM_Start(&htim12, TIM_CHANNEL_1);
/* USER CODE END 2 */

/* USER CODE BEGIN WHILE */
while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
    // 从 600 到 2400，逐步增加占空比
    for(i = 600; i < 2400; i++){
        __HAL_TIM_SetCompare(&htim12, TIM_CHANNEL_1, i); // 设置 PWM 占空比
        HAL_Delay(1); // 延时
    }

    // 从 2400 到 600，逐步减小占空比
    for(i = 2400; i > 600; i--){
        __HAL_TIM_SetCompare(&htim12, TIM_CHANNEL_1, i); // 设置 PWM 占空比
        HAL_Delay(1); // 延时
    }
}
/* USER CODE END 3 */
```

在函数中定义变量 i。

```
/* USER CODE BEGIN 1 */
uint16_t i;
/* USER CODE END 1 */
```

编写完成后编译下载到开发板中。

3.3.12 实验现象

复位后可以看到电机扩展板上的舵机在不断改变角度。

3.4 步进电机实验

3.4.1 实验目的

步进电机是一种能够将电脉冲转化为机械运动的电机，每接收一个脉冲，电机就按固定角度旋转一个步进。通过改变脉冲的频率、相位以及顺序，可以精确控制电机的位置、速度。

实验中，通过控制步进电机的输入信号，可以实现对电机的精准控制，帮助理解电机的工作方式。

3.4.2 实验内容

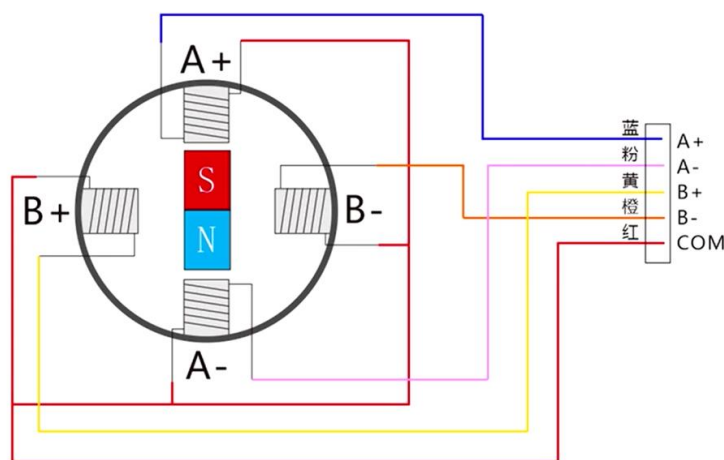
首先配置连接步进电机的四个控制引脚为输出模式，然后按照一定的时序控制这四个引脚的电平，从而驱动步进电机旋转。

本实验例程：【实验代码/4_Stepper_Motor】。

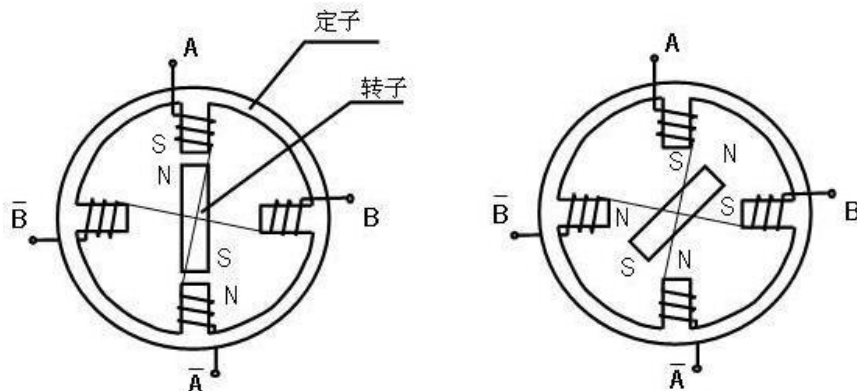
3.4.3 步进电机简介

步进电动机是一种将电脉冲信号转换成角位移或线位移的机电元件。步进电动机的输入量是脉冲序列，输出量则为相应的增量位移或步进运动。正常运动情况下，它每转一周具有固定的步数；做连续步进运动时，其旋转转速与输入脉冲的频率保持严格的对应关系(所以可以控制每个脉冲的宽度去调节速度每个脉冲转速)，由于步进电动机能直接接受数字量的控制，所以特别适宜采用微机进行控制。

四相步进电机接线图如图，该步进电机内部有四个线圈，当线圈通电时会产生感应磁场，从而驱动内部的磁铁转动。四个线圈的一端分别引出，为A、B、A-、B-四根线，另一端引出后共同接公共端COM，在使用该直流电机时，COM端接电源电压（+5V或+12V，具体需要查看该电机的手册），A、B、A-、B-四个端口依次接GND，从而形成COM——>线圈——>GND的闭合回路，驱动内部磁铁转动。



四相电机可以采用四拍驱动方式的八拍驱动方式。



接线	第 1 步	第 2 步	第 3 步	第 4 步	第 5 步	第 6 步	第 7 步	第 8 步
----	-------	-------	-------	-------	-------	-------	-------	-------

VCC	+5V	+5V	+5V	+5V	+5V	+5V	+5V	+5V
A	GND	GND						GND
B		GND	GND	GND				
A-				GND	GND	GND		
B-						GND	GND	GND

如果采用 8 步法，则步进电机的驱动依次采用上述 8 个步骤进行驱动，如果采用 4 步法，则只需要依次采用上述步骤 1、3、5、7。如果要实现反转：只要把时序倒过来就可以实现。

在本次设计中我们所用步进电机型号为 28byj-48，参数如下：

电机型号	电压 V	相数	相电阻 Ω $\pm 10\%$	步距角度	减速比	起动转矩 100P. P. S g. cm	起动频率 P. P. S	定位转矩 g. cm	摩擦转矩 g. cm	嘈声 dB	绝缘介 电强度
28BYJ-48	5	4	300	5.625/64	1:64	≥ 300	≥ 550	≥ 300	—	≤ 35	600VAC 1S

1、相数：产生不同对极 N、S 磁场的激磁线圈对数。常用 m 表示。

2、拍数：完成一个磁场周期性变化所需脉冲数或导电状态用 n 表示，或指电机转过一个齿距角所需脉冲数。

3、步距角度：对应一个脉冲信号，电机转子转过的角位移用 θ 表示。

$$\theta = \frac{360^\circ}{\text{转子齿数} \times \text{运行拍数}}$$

以常规二、四相，转子齿为 50 齿电机为例，四拍运行时步距角为 $\theta = \frac{360^\circ}{50 \times 4} = 1.8^\circ$ （俗称整步），八

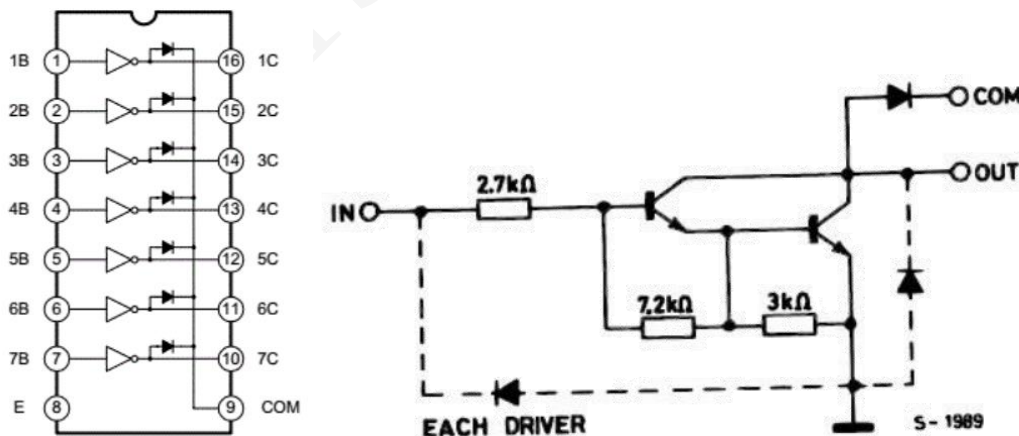
拍运行时步距角为 $\theta = \frac{360^\circ}{50 \times 8} = 0.9^\circ$ （俗称半步）。28byj-48 步进电机的转子齿数为 48，则采用四拍供电

一个脉冲信号电机转子转过的角度 $\theta = \frac{360^\circ}{48 \times 4} = 1.875^\circ$

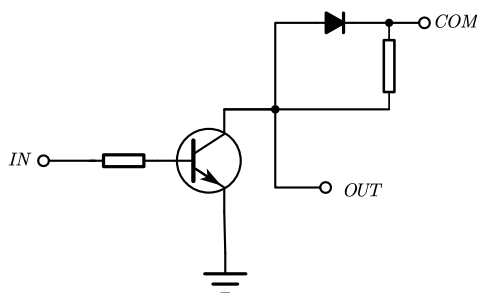
我们通过控制磁场方向的变化来改变转动方向，这时磁场转动一圈并不代表电机对应要转一圈，这里有一个参数为减速比，通过内部的转子齿来减速，所以磁场转动一圈，电机转动 $360/64=5.625$ 度。

3.4.4 ULN2003AD 达林顿晶体管

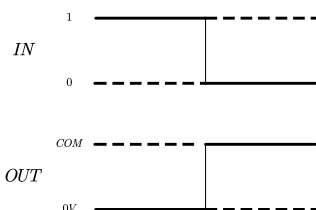
ULN2003AD 的内部原理图如下：



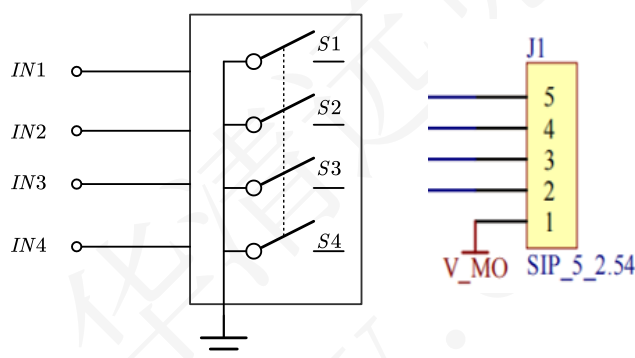
ULN2003AD 内部的每个达林顿晶体管电路如右图，该达林顿晶体管由两个三极管连接在一起，可以看作一个三极管，放大倍数是两个三极管放大倍数的乘积。当 IN 端输入高电平时，三极管导通，OUT 端电平下拉至 GND，当 IN 端输入低电平时，三极管截止，OUT 端电平等于 COM 的电平。



因此可以得出 IN 和 OUT 的时序图如图

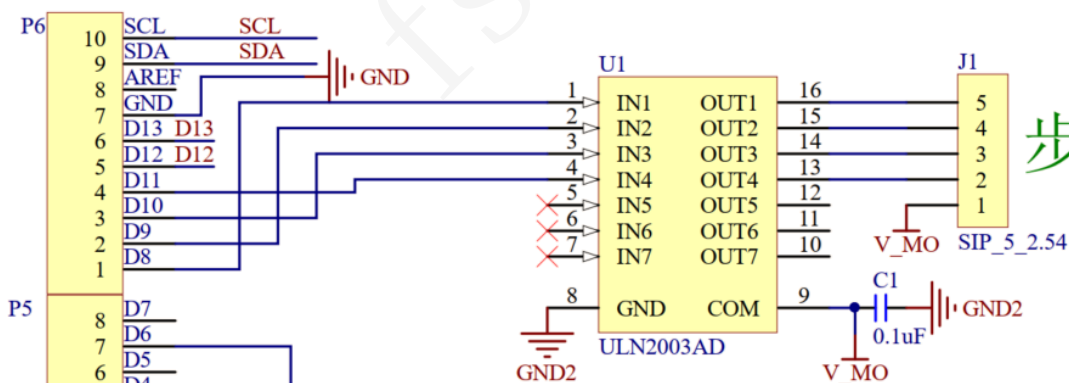


在这里达林顿晶体管就相当于一个受单片机控制的开关，单片机输出端（即 IN）输出低电平时，对应的开关 S 断开，OUT 接 COM；单片机输出端输出高电平时，对应的开关闭合 OUT 接 GND，此时电机内部对应的线圈两端有电压差，形成闭合回路。



3.4.5 硬件原理

打开电机板原理图，找到步进电机所在位置。



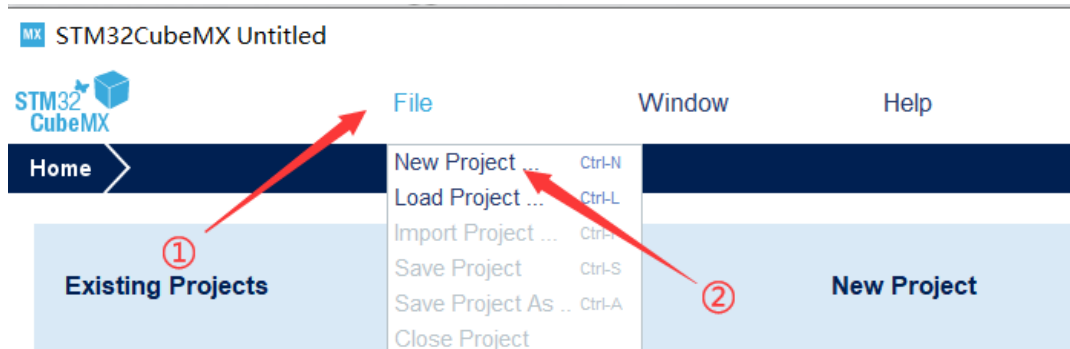
步进电机

由于单片机的引脚输出电流太低了，无法驱动大部分的设备。该原理图中，步进电机与单片机之间连接了一个 ULN2003AD 芯片，ULN2003AD 只相当于是一个开关，设备（负载）的供电是在外围电路上，而它能够通过微弱的单片机输出电流来控制外围电路的开闭，某种程度上也可以说是放大电流、增加驱动能力。

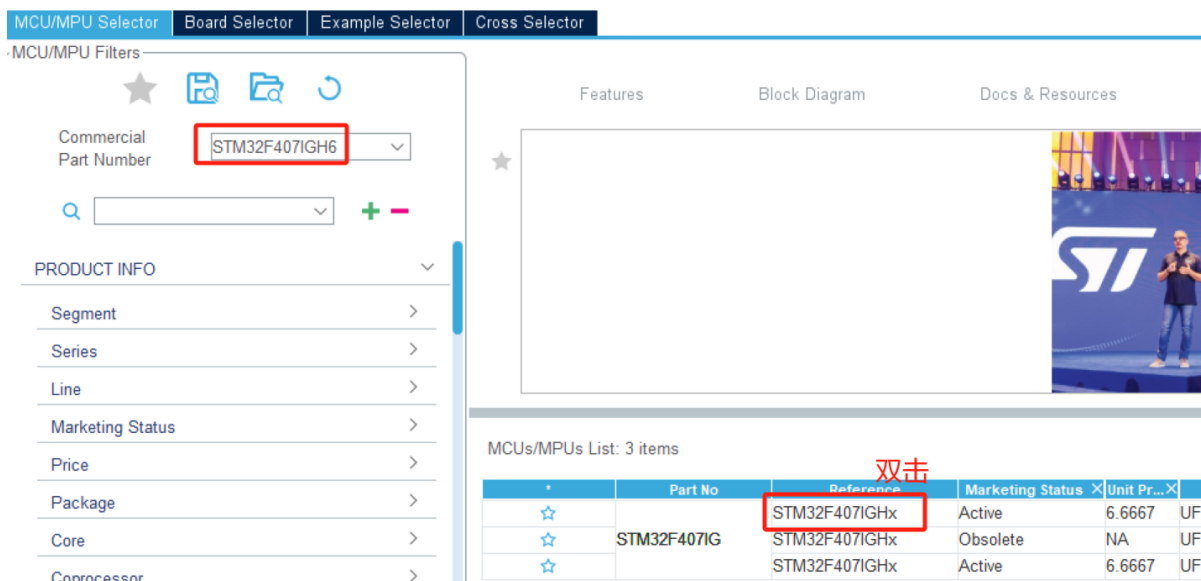
按照 3.1.5 节的方法可知，控制步进电机的四个引脚分别连接到了 STM32F407 的 PD12、PH13、PE4、PE6 上面。

3.4.6 STM32CubeMX 配置

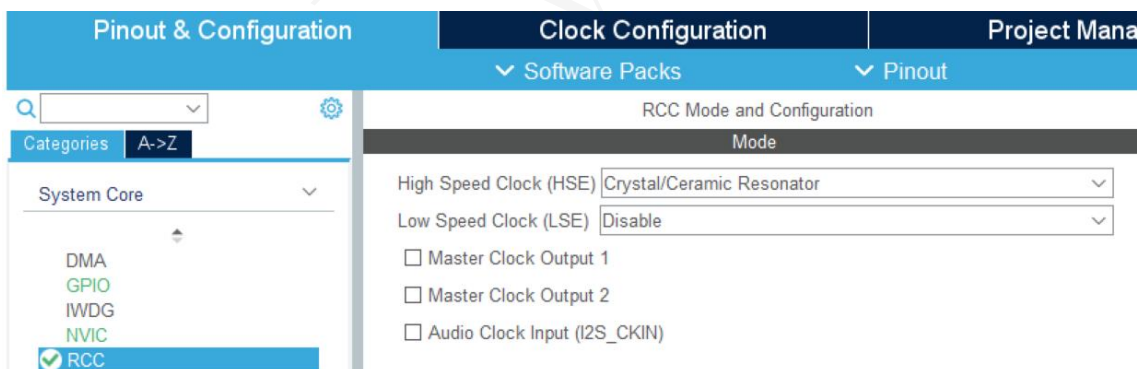
打开 STM32CubeMX->点击左上角的 File->点击 New Project。



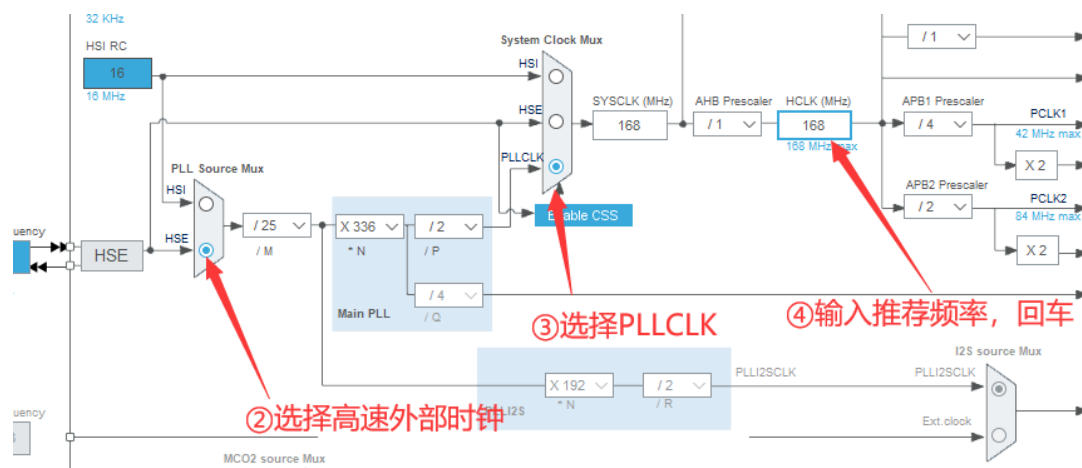
选择芯片，查看芯片的型号，在搜索框输入芯片型号，本实验中选用的芯片是 STM32F407IGHx



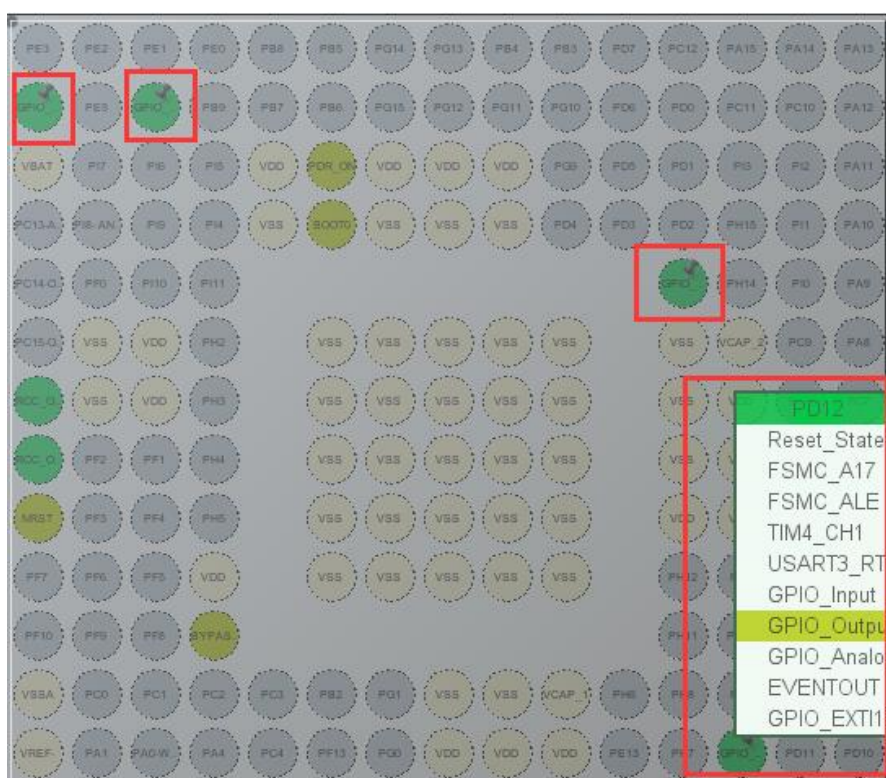
配置 RCC，点击左侧的 RCC，找到 High Speed Clock (HSE)，点击选择 Crystal/Ceramic Resonator。



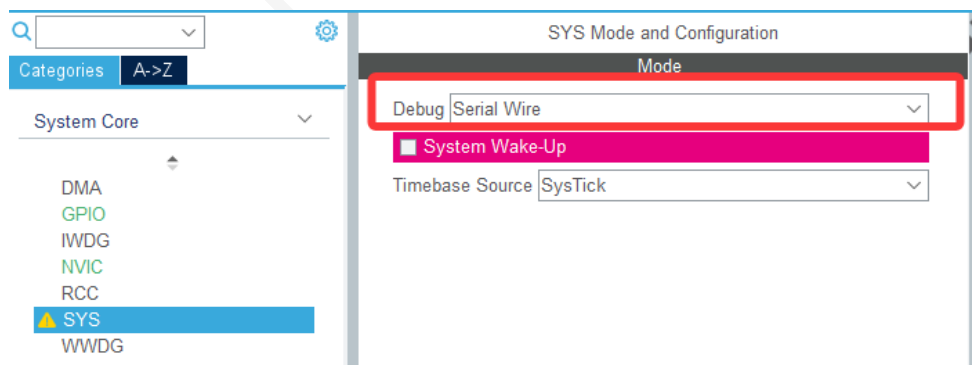
点击上方的 Clock Configuration，按下图所示配置时钟树。因为核心板上外接的晶振是 25MHz 的，时钟树上默认外接的高速晶振也是 25MHz 的，所以不用改变外接晶振频率。



配置 PD12/PE4/PE6/PH13 引脚为 GPIO_Output 模式。



将调试模式修改为 SWD 模式:



配置工程名字, 工程路径和 IDE 软件 (MDK-ARM V5.27), 注意不要有中文路径。

工程软件生成配置, 配置工程为 C 文件和 h 文件分开, 以及拷贝使用到的 HAL 库文件, 其他的文件不拷贝, 最后点击右上角的 “GENWRTE CODE”, 生成工程。

生成工程完成后会弹出提示框, 选择 Open Project 可直接从 MDK 中打开工程代码。

3.4.7 编写代码

首先在 main.c 文件的前部定义一些动作，代表对四个引脚电平的控制。

```
/* USER CODE BEGIN PV */
#define DE_A      HAL_GPIO_WritePin(GPIOD, GPIO_PIN_12,GPIO_PIN_SET);HAL_GPIO_WritePin(GPIOH, GPIO_PIN_13,GPIO_PIN_RESET);HAL_GPIO_WritePin(GPIOE, GPIO_PIN_4,GPIO_PIN_RESET);HAL_GPIO_WritePin(GPIOE, GPIO_PIN_6,GPIO_PIN_RESET)
#define DE_B      HAL_GPIO_WritePin(GPIOD, GPIO_PIN_12,GPIO_PIN_RESET);HAL_GPIO_WritePin(GPIOH, GPIO_PIN_13,GPIO_PIN_SET);HAL_GPIO_WritePin(GPIOE, GPIO_PIN_4,GPIO_PIN_RESET);HAL_GPIO_WritePin(GPIOE, GPIO_PIN_6,GPIO_PIN_RESET)
#define DE_C      HAL_GPIO_WritePin(GPIOD, GPIO_PIN_12,GPIO_PIN_RESET);HAL_GPIO_WritePin(GPIOH, GPIO_PIN_13,GPIO_PIN_RESET);HAL_GPIO_WritePin(GPIOE, GPIO_PIN_4,GPIO_PIN_SET);HAL_GPIO_WritePin(GPIOE, GPIO_PIN_6,GPIO_PIN_RESET)
#define DE_D      HAL_GPIO_WritePin(GPIOD, GPIO_PIN_12,GPIO_PIN_RESET);HAL_GPIO_WritePin(GPIOH, GPIO_PIN_13,GPIO_PIN_RESET);HAL_GPIO_WritePin(GPIOE, GPIO_PIN_4,GPIO_PIN_RESET);HAL_GPIO_WritePin(GPIOE, GPIO_PIN_6,GPIO_PIN_SET)

#define DE_AB      HAL_GPIO_WritePin(GPIOD, GPIO_PIN_12,GPIO_PIN_SET);HAL_GPIO_WritePin(GPIOH, GPIO_PIN_13,GPIO_PIN_SET);HAL_GPIO_WritePin(GPIOE, GPIO_PIN_4,GPIO_PIN_RESET);HAL_GPIO_WritePin(GPIOE, GPIO_PIN_6,GPIO_PIN_RESET)
#define DE_BC      HAL_GPIO_WritePin(GPIOD, GPIO_PIN_12,GPIO_PIN_RESET);HAL_GPIO_WritePin(GPIOH, GPIO_PIN_13,GPIO_PIN_SET);HAL_GPIO_WritePin(GPIOE, GPIO_PIN_4,GPIO_PIN_SET);HAL_GPIO_WritePin(GPIOE, GPIO_PIN_6,GPIO_PIN_RESET)
#define DE_CD      HAL_GPIO_WritePin(GPIOD, GPIO_PIN_12,GPIO_PIN_RESET);HAL_GPIO_WritePin(GPIOH, GPIO_PIN_13,GPIO_PIN_RESET);HAL_GPIO_WritePin(GPIOE, GPIO_PIN_4,GPIO_PIN_SET);HAL_GPIO_WritePin(GPIOE, GPIO_PIN_6,GPIO_PIN_SET)
#define DE_DA      HAL_GPIO_WritePin(GPIOD, GPIO_PIN_12,GPIO_PIN_SET);HAL_GPIO_WritePin(GPIOH, GPIO_PIN_13,GPIO_PIN_RESET);HAL_GPIO_WritePin(GPIOE, GPIO_PIN_4,GPIO_PIN_RESET);HAL_GPIO_WritePin(GPIOE, GPIO_PIN_6,GPIO_PIN_SET)
/* USER CODE END PV */
```

然后在主循环中依次执行这几个操作。

```
/* USER CODE BEGIN WHILE */
while (1)
{
    /******八拍方式*****/
    DE_A;
    HAL_Delay(3);//可进行调速，延时时间不能太短
    DE_AB;
    HAL_Delay(3);
    DE_B;
    HAL_Delay(3);
    DE_BC;
    HAL_Delay(3);
    DE_C;
    HAL_Delay(3);
    DE_CD;
```

```
HAL_Delay(3);
DE_D;
HAL_Delay(3);
DE_DA;
HAL_Delay(3);
/* USER CODE END WHILE */
```

编写完成后编译下载到开发板中。

3.4.8 实验现象

复位后可以看到步进电机在旋转。

3.5 USART1 串口数据收发实验（中断模式）

3.5.1 实验目的

实验中配置了 USART1 串口，通过串口进行数据的接收与发送。通过学习如何在 STM32 中设置串口通信（包括波特率、数据位、停止位等），掌握基本的串口通信原理。

通过本实验，学习如何使用 STM32 的中断机制来接收串口数据。实验中使用了 HAL_UART_Receive_IT 函数进行串口数据的中断接收，并在中断回调函数中处理接收到的数据。

中断方式相较于轮询方式更加高效，可以在数据到达时立即响应，适合实时性要求较高的应用。

通过本实验，还可以学会如何利用 printf() 函数进行串口数据的打印。

3.5.2 实验内容

实验内容主要围绕 STM32 开发板上的串口通信和中断机制进行，具体包括串口数据接收、串口中断处理、数据回传以及使用 printf 进行串口调试输出等操作。

本实验例程：【实验代码/5_USART1】。

3.5.3 串行通信简介

3.5.3.1 常见的串行通信协议

通信协议定义了数据传输的格式、速率、时序和控制信号等细节。只有两个设备之间遵循相同的协议才能按照约定的方式进行通信。以下是一些常见的串行通信协议。

RS-232 是一种最常见的串行通信协议之一，用于在计算机和外部设备之间传输数据。它定义了电气特性和信号线连接方式，而没有定义数据帧格式。RS-232 通常使用 DB-9 或 DB-25 连接器，并且支持相对较短的通信距离，最远为十几米。



RS-485 是一种用于在工业环境中进行远距离数据传输的串行通信协议，传输距离可达一千米以上。与 RS-232 相比，RS-485 具有更好的抗干扰性和更长的通信距离，可支持多个设备的多点通信。它通常用于工业自动化和建筑自动化等领域。